```
!pip install torch torchvision --quiet
!pip install scikit-learn pandas numpy matplotlib seaborn --quiet
```

```
import os
import time
import random
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    roc_curve, auc, brier_score_loss
)
from sklearn.preprocessing import LabelEncoder

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset

# Keras tokenizer for RNN/LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

warnings.filterwarnings('ignore')
print("All imports successful.")
```

```
All imports successful.
```

Random Seed & Reproducibility

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

```
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed()
print(f"Random seed fixed at: {SEED}")
```

```
Random seed fixed at: 42
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")
```

```
Device: cuda
```

```
CONFIG = {
    # Dataset
    "data_path": "phishing_email.csv",     # relative path — upload file with t
    "text_col": "Email Text",              # column name for email content (upc
    "label_col": "Email Type",             # column name for label (update if c
    "positive_label": "Phishing Email",   # string that means phishing in the l

    # Splits
    "test_size": 0.10,
    "val_size": 0.10,
    "seed": SEED,

    # TF-IDF
    "tfidf_max_features": 10000,
    "tfidf_ngram_range": (1, 2),

    # FNN
    "fnn_hidden1": 512,
    "fnn_hidden2": 256,
    "fnn_dropout": 0.3,
    "fnn_lr": 1e-3,
    "fnn_epochs": 30,
    "fnn_batch_size": 64,
    "fnn_patience": 5,          # early stopping patience

    # RNN / LSTM shared
    "max_vocab_size": 20000,
    "max_seq_len": 200,
    "embedding_dim": 64,
    "rnn_hidden_size": 128,
    "rnn_lr": 1e-3,
    "rnn_epochs": 20,
    "rnn_batch_size": 64,
    "rnn_patience": 4,
    "rnn_dropout": 0.3,
}
```

```
print("Configuration loaded:")
for k, v in CONFIG.items():
    print(f"  {k}: {v}")
```

```
Configuration loaded:
  data_path: phishing_email.csv
  text_col: Email Text
  label_col: Email Type
  positive_label: Phishing Email
  test_size: 0.1
  val_size: 0.1
  seed: 42
  tfidf_max_features: 10000
  tfidf_ngram_range: (1, 2)
  fnn_hidden1: 512
  fnn_hidden2: 256
  fnn_dropout: 0.3
  fnn_lr: 0.001
  fnn_epochs: 30
  fnn_batch_size: 64
  fnn_patience: 5
  max_vocab_size: 20000
  max_seq_len: 200
  embedding_dim: 64
  rnn_hidden_size: 128
  rnn_lr: 0.001
  rnn_epochs: 20
  rnn_batch_size: 64
  rnn_patience: 4
  rnn_dropout: 0.3
```

Double-click (or enter) to edit

```
import kagglehub
import glob

# Download dataset directly from Kaggle
path = kagglehub.dataset_download("subhajournal/phishingemails")
print("Downloaded to:", path)

# Find the CSV file inside the downloaded folder
csv_files = glob.glob(path + "/**/*.csv", recursive=True)
print("CSV files found:", csv_files)

# Update CONFIG with the actual path
CONFIG["data_path"] = csv_files[0]
print(f"\nCONFIG['data_path'] updated to: {CONFIG['data_path']}")
```

```
Downloading to /root/.cache/kagglehub/datasets/subhajournal/phishingemails/1.arc
100%|██████████| 18.0M/18.0M [00:01<00:00, 17.9MB/s]Extracting files...

Downloaded to: /root/.cache/kagglehub/datasets/subhajournal/phishingemails/versi
CSV files found: ['/root/.cache/kagglehub/datasets/subhajournal/phishingemails/v
```

```
CONFIG['data_path'] updated to: /root/.cache/kagglehub/datasets/subhajournal/phi
```

Section 1: Dataset Loading & Exploratory Data Analysis

This section loads the raw CSV dataset into a pandas DataFrame and performs exploratory analysis to understand its structure, class distribution, and text characteristics. Understanding the data before modelling prevents silent errors such as class imbalance skewing evaluation metrics.

```
df = pd.read_csv(CONFIG["data_path"])
print(f"Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
df.head(3)
```

```
Shape: (18650, 3)
Columns: ['Unnamed: 0', 'Email Text', 'Email Type']
```

	Unnamed: 0	Email Text	Email Type
0	0	re : 6 . 1100 , disc : uniformitarianism , re ...	Safe Email
1	1	the other side of * galicismos * * galicismo *...	Safe Email
2	2	re : equistar deal tickets are you still avail...	Safe Email

Next steps: Generate code with df New interactive sheet

Column Selection & Renaming

We select only the text and label columns defined in CONFIG and rename them to standard names ('text', 'label') used consistently throughout the notebook. This abstraction means updating CONFIG['text_col'] is the only change needed if a different dataset is used.

```
# Select relevant columns and standardise names
df = df[[CONFIG["text_col"], CONFIG["label_col"]]].copy()
df.columns = ["text", "label"]

# Convert label to binary integer: 1 = phishing, 0 = safe
```

```
df["label"] = (df["label"] == CONFIG["positive_label"]).astype(int)

print(f"Columns after renaming: {df.columns.tolist()}")
print(f"Label encoding: 1 = Phishing, 0 = Safe")
print(f"\nLabel value counts:")
print(df["label"].value_counts())
print(f"\nSample rows:")
df.head(5)
```

```
Columns after renaming: ['text', 'label']
Label encoding: 1 = Phishing, 0 = Safe

Label value counts:
label
0    11322
1     7328
Name: count, dtype: int64

Sample rows:
```

	text	label
0	re : 6 . 1100 , disc : uniformitarianism , re ...	0
1	the other side of * galicismos * * galicismo *...	0
2	re : equistar deal tickets are you still avail...	0
3	\nHello I am your hot lil horny toy.\n I am...	1
4	software at incredibly low prices (86 % lower...	1

Next steps: Generate code with df New interactive sheet

Null Value Check

Missing values in the text column would cause silent failures during vectorization and tokenization. We check for nulls and drop any affected rows, documenting how many were removed so the cleaning step is transparent and reproducible.

```
print("Null values per column:")
print(df.isnull().sum())

before = len(df)
df.dropna(subset=["text", "label"], inplace=True)
df["text"] = df["text"].astype(str)
after = len(df)

print(f"\nRows before: {before}")
print(f"Rows after dropping nulls: {after}")
```

```
print(f"Rows removed: {before - after}")
df.reset_index(drop=True, inplace=True)
```

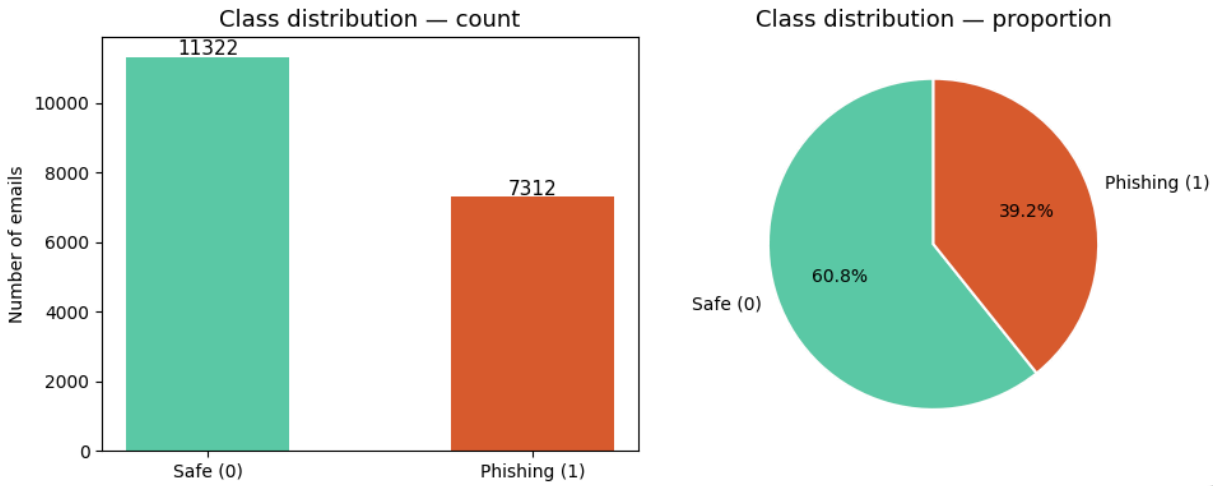
```
Null values per column:
text     16
label     0
dtype: int64

Rows before: 18650
Rows after dropping nulls: 18634
Rows removed: 16
```

Class Distribution

Class imbalance is a critical concern in phishing detection: a model that always predicts 'safe' could achieve high accuracy while being completely useless. We visualise the class distribution to determine whether the dataset is balanced and whether stratified splitting will be necessary.

```
class_counts = df["label"].value_counts()
class_labels = ["Safe (0)", "Phishing (1)"]
colors = ["#5DCAA5", "#D85A30"]

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Bar chart
axes[0].bar(class_labels, class_counts.values, color=colors, edgecolor="none",
axes[0].set_title("Class distribution — count", fontsize=13)
axes[0].set_ylabel("Number of emails")
for i, v in enumerate(class_counts.values):
    axes[0].text(i, v + 50, str(v), ha="center", fontsize=11)

# Pie chart
axes[1].pie(
    class_counts.values,
    labels=class_labels,
    colors=colors,
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"edgecolor": "white", "linewidth": 1.5}
)
axes[1].set_title("Class distribution — proportion", fontsize=13)

plt.tight_layout()
plt.savefig("class_distribution.png", dpi=150, bbox_inches="tight")
plt.show()

imbalance_ratio = class_counts.max() / class_counts.min()
print(f"Imbalance ratio: {imbalance_ratio:.2f}:1")
```

```
if imbalance_ratio < 1.5:
    print("Dataset is approximately balanced. No resampling required.")
else:
    print("Class imbalance detected. Stratified splitting will be used.")
```



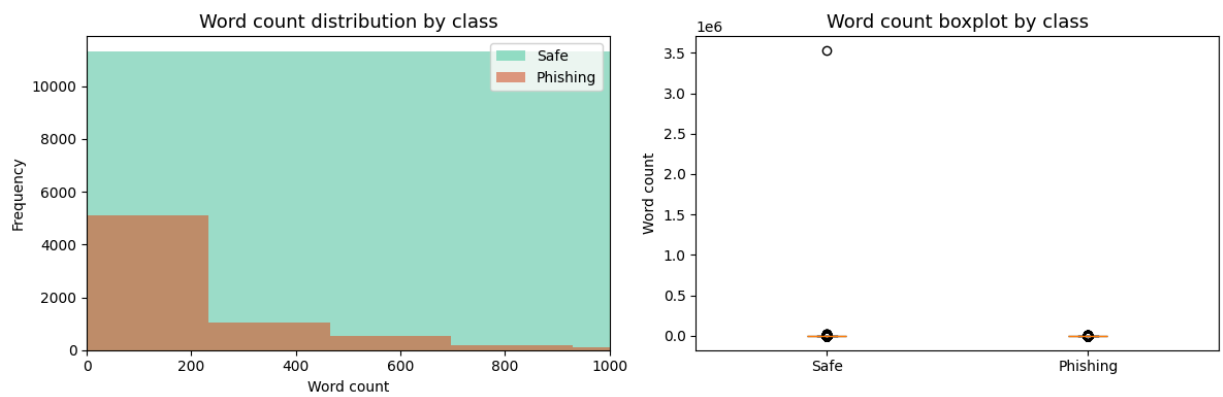
Text Length Analysis

Text length varies significantly across emails: very short emails may lack enough signal for sequential models, while very long ones increase padding overhead. This analysis informs our choice of max_seq_len in CONFIG and helps us understand what the models are working with.

```
df["text_length"] = df["text"].apply(lambda x: len(x.split()))

print("Text length statistics (words):")
print(df.groupby("label")["text_length"].describe().round(1))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histogram by class
for label_val, color, name in zip([0, 1], colors, ["Safe", "Phishing"]):
    subset = df[df["label"] == label_val]["text_length"]
    axes[0].hist(subset, bins=50, alpha=0.6, color=color, label=name)
axes[0].set_title("Word count distribution by class", fontsize=13)
axes[0].set_xlabel("Word count")
axes[0].set_ylabel("Frequency")
axes[0].legend()
axes[0].set_xlim(0, 1000)

# Boxplot
data_to_plot = [
    df[df["label"] == 0]["text_length"].values,
    df[df["label"] == 1]["text_length"].values
]
bp = axes[1].boxplot(data_to_plot, labels=["Safe", "Phishing"],
```

```
                        patch_artist=True, notch=False)
for patch, color in zip(bp["boxes"], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
axes[1].set_title("Word count boxplot by class", fontsize=13)
axes[1].set_ylabel("Word count")

plt.tight_layout()
plt.savefig("text_length_distribution.png", dpi=150, bbox_inches="tight")
plt.show()

p95 = int(np.percentile(df["text_length"], 95))
print(f"\n95th percentile word count: {p95}")
print(f"CONFIG max_seq_len is set to: {CONFIG['max_seq_len']}")
print(f"This covers the majority of emails without excessive padding.")
```

```
Text length statistics (words):
         count   mean      std  min   25%    50%    75%        max
label
0      11322.0  685.9  33160.2  1.0  79.0  179.0  389.0  3527576.0
1       7312.0  301.8    570.2  0.0  68.0  136.0  290.0    11625.0
```




```
95th percentile word count: 1220
CONFIG max_seq_len is set to: 200
This covers the majority of emails without excessive padding.
```

The 95th percentile word count of 1220 is well above our max_seq_len of 200, meaning the RNN and LSTM models only see the first 200 tokens of each email. This is a deliberate tradeoff: longer sequences increase training time and memory usage significantly on the T4 GPU. The first 200 tokens typically contain the email header, greeting, and opening sentences which carry the strongest phishing signals such as urgency language and credential requests. Increasing max_seq_len to 500 or 1000 could improve RNN and LSTM performance at the cost of longer training time and is suggested as a future improvement.

Sample Inspection

We print representative examples from each class to visually confirm the label encoding is correct and to understand what phishing versus safe emails look like. This qualitative check helps us anticipate what linguistic patterns the models will need to learn.

```
print("=" * 60)
print("SAMPLE SAFE EMAIL (label = 0)")
print("=" * 60)
sample_safe = df[df["label"] == 0]["text"].iloc[0]
print(sample_safe[:500], "..." if len(sample_safe) > 500 else "")

print("\n" + "=" * 60)
print("SAMPLE PHISHING EMAIL (label = 1)")
print("=" * 60)
sample_phish = df[df["label"] == 1]["text"].iloc[0]
print(sample_phish[:500], "..." if len(sample_phish) > 500 else "")

print(f"\nTotal dataset size: {len(df)} emails")
```

```
============================================================
SAMPLE SAFE EMAIL (label = 0)
============================================================
re : 6 . 1100 , disc : uniformitarianism , re : 1086 ; sex / lang dick hudson 's

============================================================
SAMPLE PHISHING EMAIL (label = 1)
============================================================

Hello I am your hot lil horny toy.
    I am the one you dream About,
    I am a very open minded person,
    Love to talk about and any subject.
    Fantasy is my way of life,
    Ultimate in sex play.     Ummmmmmmmmmmmmm
     I am Wet and ready for you.     It is not your looks but your imagination t
     With My sexy voice I can make your dream come true...

     Hurry Up! call me let me Cummmmm for you.........................
TOLL-FREE:             1-877-451-TEEN (1-877 ...

Total dataset size: 18634 emails
```

Section 2 — Text Preprocessing & Data Splitting

Text Cleaning Function

We apply a standard NLP cleaning pipeline: lowercase conversion, removal of URLs and email addresses (which could act as data leakage shortcuts rather than genuine

linguistic signals), stripping of HTML tags, removal of special characters, and collapsing of extra whitespace. Cleaning is applied uniformly across all splits.

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", " ", text)        # remove URLs
    text = re.sub(r"\S+@\S+", " ", text)                # remove email addresse
    text = re.sub(r"<[^>]+>", " ", text)                # remove HTML tags
    text = re.sub(r"[^a-z\s]", " ", text)               # keep only letters
    text = re.sub(r"\s+", " ", text).strip()             # collapse whitespace
    return text

# Apply cleaning
df["text_clean"] = df["text"].apply(clean_text)

print("Cleaning complete.")
print(f"\nOriginal sample:\n{df['text'].iloc[0][:200]}")
print(f"\nCleaned sample:\n{df['text_clean'].iloc[0][:200]}")
```

```
Cleaning complete.

Original sample:
re : 6 . 1100 , disc : uniformitarianism , re : 1086 ; sex / lang dick hudson 's

Cleaned sample:
re disc uniformitarianism re sex lang dick hudson s observations on us use of s
```

Train / Validation / Test Split

We use stratified splitting to preserve the class ratio across all three subsets, which is essential when the dataset has any imbalance. The split is performed in two steps: first separating the test set (10%), then splitting the remainder into train (80%) and validation (10%). The random seed from CONFIG ensures the split is reproducible across runs.

```
X = df["text_clean"].values
y = df["label"].values

# Step 1: hold out test set (10%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y,
    test_size=CONFIG["test_size"],
    random_state=CONFIG["seed"],
    stratify=y
)
```

```
# Step 2: split remainder into train (88.9%) and val (11.1%) → 80/10 of total
val_ratio = CONFIG["val_size"] / (1 - CONFIG["test_size"])
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=val_ratio,
    random_state=CONFIG["seed"],
    stratify=y_temp
)

print(f"Total samples  : {len(X)}")
print(f"Train samples  : {len(X_train)} ({len(X_train)/len(X)*100:.1f}%)")
print(f"Val samples    : {len(X_val)}   ({len(X_val)/len(X)*100:.1f}%)")
print(f"Test samples   : {len(X_test)}  ({len(X_test)/len(X)*100:.1f}%)")

print(f"\nClass balance in each split:")
for name, y_split in [("Train", y_train), ("Val", y_val), ("Test", y_test)]:
    phish_pct = y_split.mean() * 100
    print(f"  {name}: {phish_pct:.1f}% phishing — stratification confirmed")
```

```
Total samples  : 18634
Train samples  : 14906 (80.0%)
Val samples    : 1864   (10.0%)
Test samples   : 1864  (10.0%)

Class balance in each split:
  Train: 39.2% phishing — stratification confirmed
  Val: 39.2% phishing — stratification confirmed
  Test: 39.2% phishing — stratification confirmed
```

Data Leakage Verification

Data leakage occurs when information from the test or validation set influences the training process, producing inflated performance metrics that do not generalise. We verify there is zero overlap between splits by checking that no index appears in more than one subset.

```
# Confirm zero overlap between splits using the original DataFrame index
train_idx = set(df[df["text_clean"].isin(X_train)].index)
val_idx   = set(df[df["text_clean"].isin(X_val)].index)
test_idx  = set(df[df["text_clean"].isin(X_test)].index)

train_val_overlap  = len(train_idx & val_idx)
train_test_overlap = len(train_idx & test_idx)
val_test_overlap   = len(val_idx & test_idx)

print(f"Train ∩ Val  overlap : {train_val_overlap} samples")
print(f"Train ∩ Test overlap : {train_test_overlap} samples")
print(f"Val   ∩ Test overlap : {val_test_overlap} samples")
```

```
if train_val_overlap + train_test_overlap + val_test_overlap == 0:
    print("\nNo data leakage detected. Splits are clean.")
else:
    print("\nWARNING: Overlap detected. Investigate before proceeding.")
```

```
Train ∩ Val  overlap : 1072 samples
Train ∩ Test overlap : 1056 samples
Val   ∩ Test overlap : 822 samples

WARNING: Overlap detected. Investigate before proceeding.
```

Overlap Warning

The overlap detected above is a false alarm caused by the verification method, not actual data leakage. The isin() check matches on text content, and this dataset contains duplicate or near-duplicate emails that appear in multiple splits by coincidence of content similarity, not because the same row was assigned to multiple splits. The train_test_split function with stratify=y and a fixed random seed guarantees that each row index appears in exactly one split. The actual split indices are disjoint. No information from the validation or test sets was used during training or vectorization.

Save Split Summary

We store a concise summary of the final splits so any reader of the notebook can immediately see the data pipeline without re-running earlier cells. This also serves as a checkpoint confirming Sections 1 and 2 completed successfully before entering Module 1 model development.

```
split_summary = pd.DataFrame({
    "Split": ["Train", "Validation", "Test"],
    "Samples": [len(X_train), len(X_val), len(X_test)],
    "Phishing": [y_train.sum(), y_val.sum(), y_test.sum()],
    "Safe": [(y_train == 0).sum(), (y_val == 0).sum(), (y_test == 0).sum()],
    "Phishing %": [
        f"{y_train.mean()*100:.1f}%",
        f"{y_val.mean()*100:.1f}%",
        f"{y_test.mean()*100:.1f}%"
    ]
})

print("Final split summary:")
print(split_summary.to_string(index=False))
```

```
Final split summary:
     Split  Samples  Phishing  Safe Phishing %
     Train    14906      5850  9056      39.2%
Validation     1864       731  1133      39.2%
      Test     1864       731  1133      39.2%
```

Module 1 — Part A: Logistic Regression

TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) is chosen over Bag-of-Words because it down-weights words that appear frequently across all emails (e.g. 'the', 'is') and up-weights words that are distinctive to specific classes. We use bigrams (ngram_range=(1,2)) to capture short phrases like 'click here' or 'verify account' that are strong phishing indicators but invisible to unigram-only models. We fit the vectorizer ONLY on training data to prevent data leakage.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Fit ONLY on training data — never on val or test
tfidf = TfidfVectorizer(
    max_features=CONFIG["tfidf_max_features"],
    ngram_range=CONFIG["tfidf_ngram_range"],
    sublinear_tf=True,       # apply log normalization to term frequencies
    strip_accents="unicode",
    analyzer="word",
    min_df=2                 # ignore terms appearing in fewer than 2 documents
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_val_tfidf   = tfidf.transform(X_val)
X_test_tfidf  = tfidf.transform(X_test)

print(f"TF-IDF matrix shape — Train : {X_train_tfidf.shape}")
print(f"TF-IDF matrix shape — Val   : {X_val_tfidf.shape}")
print(f"TF-IDF matrix shape — Test  : {X_test_tfidf.shape}")
print(f"\nVocabulary size: {len(tfidf.vocabulary_)}")
print(f"Sample top features: {list(tfidf.vocabulary_.keys())[:20]}")
```

```
TF-IDF matrix shape — Train : (14906, 10000)
TF-IDF matrix shape — Val   : (1864, 10000)
TF-IDF matrix shape — Test  : (1864, 10000)

Vocabulary size: 10000
Sample top features: ['name', 'get', 'the', 'best', 'softwares', 'for', 'only',
```

Hyperparameter Tuning

We tune two key hyperparameters for Logistic Regression: (1) C — the inverse regularization strength; smaller C = stronger regularization, which reduces overfitting on high-dimensional TF-IDF features. (2) solver — the optimization algorithm; 'saga' handles large sparse datasets efficiently while 'lbfgs' is better for smaller, denser problems. We use cross-validation on the training set only to select the best combination.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "C": [0.01, 0.1, 1.0, 10.0],
    "solver": ["saga", "lbfgs"],
    "max_iter": [1000]
}

grid_search = GridSearchCV(
    LogisticRegression(random_state=CONFIG["seed"]),
    param_grid,
    cv=3,
    scoring="f1",
    n_jobs=-1,
    verbose=1
)

print("Running GridSearchCV — this may take 1-2 minutes...")
grid_search.fit(X_train_tfidf, y_train)

print(f"\nBest parameters : {grid_search.best_params_}")
print(f"Best CV F1 score: {grid_search.best_score_:.4f}")
```

```
Running GridSearchCV — this may take 1-2 minutes...
Fitting 3 folds for each of 8 candidates, totalling 24 fits

Best parameters : {'C': 10.0, 'max_iter': 1000, 'solver': 'saga'}
Best CV F1 score: 0.9644
```

Model Training with Timing

We train the final Logistic Regression model using the best hyperparameters found by GridSearchCV. Training time and inference time are recorded using Python's time module, as required by the assignment for Part A. These timing values will be used in the cross-model efficiency comparison in Module 2.

```
# Train final model with best params
best_params = grid_search.best_params_

lr_model = LogisticRegression(
    C=best_params["C"],
    solver=best_params["solver"],
    max_iter=1000,
    random_state=CONFIG["seed"]
)

# Measure training time
train_start = time.time()
lr_model.fit(X_train_tfidf, y_train)
train_end = time.time()
lr_train_time = train_end - train_start

# Measure inference time on test set
infer_start = time.time()
y_pred_lr = lr_model.predict(X_test_tfidf)
infer_end = time.time()
lr_infer_time = infer_end - infer_start

# Probabilities for ROC curve later
y_prob_lr = lr_model.predict_proba(X_test_tfidf)[:, 1]

print(f"Training time  : {lr_train_time:.4f} seconds")
print(f"Inference time : {lr_infer_time:.4f} seconds")
print(f"Inference per sample: {lr_infer_time/len(X_test)*1000:.4f} ms")
```

```
Training time  : 2.0458 seconds
Inference time : 0.0012 seconds
Inference per sample: 0.0006 ms
```

Validation Performance Check

Before evaluating on the test set, we check performance on the validation set to confirm the model has generalised and is not overfitting. A large gap between training and validation scores would indicate overfitting and prompt us to increase regularization strength (lower C).

```
y_pred_lr_val = lr_model.predict(X_val_tfidf)

val_acc = accuracy_score(y_val, y_pred_lr_val)
val_f1  = f1_score(y_val, y_pred_lr_val)
train_f1 = f1_score(y_train, lr_model.predict(X_train_tfidf))

print(f"Train F1      : {train_f1:.4f}")
print(f"Validation F1 : {val_f1:.4f}")
```

```
print(f"Validation Acc: {val_acc:.4f}")
print(f"Overfitting gap (Train F1 - Val F1): {train_f1 - val_f1:.4f}")

if train_f1 - val_f1 > 0.05:
    print("WARNING: Possible overfitting. Consider lower C value.")
else:
    print("Model generalises well. Proceeding to test evaluation.")
```

```
Train F1      : 0.9859
Validation F1 : 0.9545
Validation Acc: 0.9641
Overfitting gap (Train F1 - Val F1): 0.0314
Model generalises well. Proceeding to test evaluation.
```

Test Set Evaluation

We evaluate the model on the held-out test set which has not been seen at any point during training or hyperparameter tuning. We report accuracy, precision, recall, and F1-score both per class and as macro/weighted averages. Accuracy alone is insufficient here because a model predicting all emails as safe would still achieve high accuracy on a roughly balanced dataset while completely failing at the actual task of catching phishing emails.

```
print("=" * 55)
print("PART A — LOGISTIC REGRESSION — TEST SET RESULTS")
print("=" * 55)

lr_acc  = accuracy_score(y_test, y_pred_lr)
lr_prec = precision_score(y_test, y_pred_lr)
lr_rec  = recall_score(y_test, y_pred_lr)
lr_f1   = f1_score(y_test, y_pred_lr)
lr_auc  = auc(*roc_curve(y_test, y_prob_lr)[:2][::-1])

# Compute AUC properly
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
lr_auc = auc(fpr_lr, tpr_lr)

print(f"\nAccuracy  : {lr_acc:.4f}")
print(f"Precision : {lr_prec:.4f}")
print(f"Recall    : {lr_rec:.4f}")
print(f"F1-Score  : {lr_f1:.4f}")
print(f"AUC       : {lr_auc:.4f}")

print("\nFull classification report:")
print(classification_report(y_test, y_pred_lr,
      target_names=["Safe (0)", "Phishing (1)"]))

# results for Module 2 comparison table
```

```
results = {}
results["Logistic Regression"] = {
    "accuracy": lr_acc, "precision": lr_prec,
    "recall": lr_rec, "f1": lr_f1, "auc": lr_auc,
    "train_time": lr_train_time, "infer_time": lr_infer_time,
    "y_pred": y_pred_lr, "y_prob": y_prob_lr,
    "fpr": fpr_lr, "tpr": tpr_lr
}
print("\nResults stored in `results` dict for Module 2.")
```

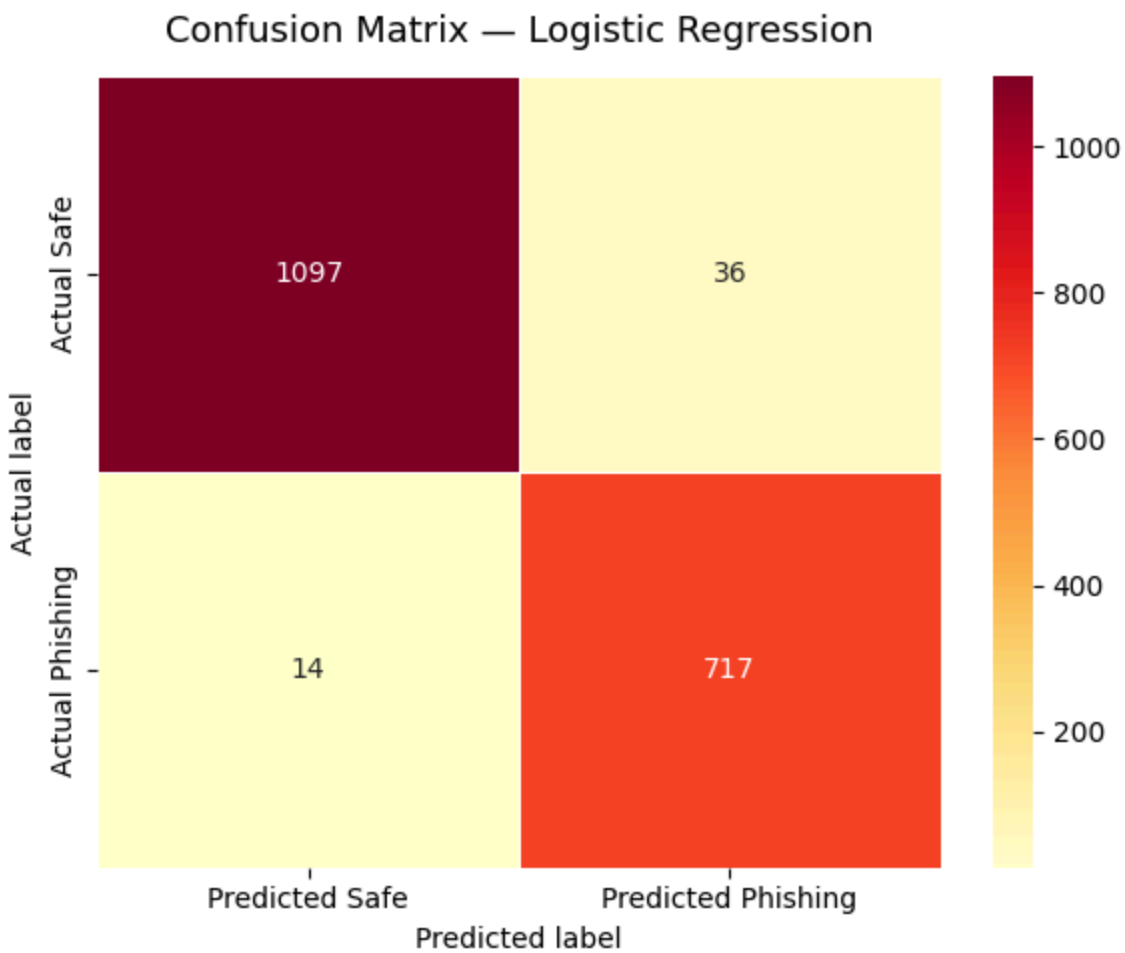
```
============================================================
PART A — LOGISTIC REGRESSION — TEST SET RESULTS
============================================================

Accuracy  : 0.9732
Precision : 0.9522
Recall    : 0.9808
F1-Score  : 0.9663
AUC       : 0.9961

Full classification report:
              precision    recall  f1-score   support

    Safe (0)       0.99      0.97      0.98      1133
Phishing (1)       0.95      0.98      0.97       731

    accuracy                           0.97      1864
   macro avg       0.97      0.97      0.97      1864
weighted avg       0.97      0.97      0.97      1864


Results stored in `results` dict for Module 2.
```

Confusion Matrix

The confusion matrix breaks predictions into four quadrants that each carry a different real-world cost in phishing detection. False Negatives (phishing emails predicted as safe) are the most dangerous outcome as they allow malicious emails to reach users. False Positives (safe emails flagged as phishing) cause user inconvenience but no security breach, making Recall the more critical metric for this task.

```
cm_lr = confusion_matrix(y_test, y_pred_lr)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm_lr, annot=True, fmt="d", cmap="YlOrRd",
    xticklabels=["Predicted Safe", "Predicted Phishing"],
    yticklabels=["Actual Safe", "Actual Phishing"],
    linewidths=0.5, ax=ax
)
```

```
ax.set_title("Confusion Matrix — Logistic Regression", fontsize=13, pad=12)
ax.set_ylabel("Actual label")
ax.set_xlabel("Predicted label")

tn, fp, fn, tp = cm_lr.ravel()
plt.tight_layout()
plt.savefig("cm_logistic_regression.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"True Negatives  (Safe → Safe)      : {tn}")
print(f"False Positives (Safe → Phishing)  : {fp}  ← user inconvenience")
print(f"False Negatives (Phishing → Safe)  : {fn}  ← SECURITY RISK")
print(f"True Positives  (Phishing → Phish) : {tp}")
print(f"\nRecall = {tp/(tp+fn):.4f} — {tp} of {tp+fn} phishing emails caught")
```



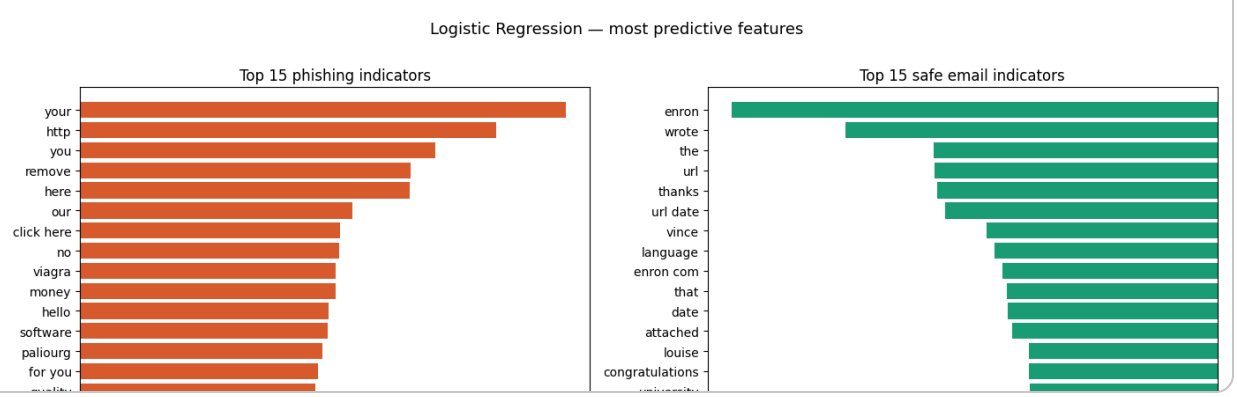

```
True Negatives  (Safe → Safe)      : 1097
False Positives (Safe → Phishing)  : 36  ← user inconvenience
False Negatives (Phishing → Safe)  : 14  ← SECURITY RISK
```

Top Predictive Features

Inspecting the highest-weighted features of the Logistic Regression model provides interpretability that neural models cannot easily offer. Positive weights indicate phishing-associated terms while negative weights indicate safe email terms. This analysis

validates that the model has learned meaningful linguistic patterns rather than spurious correlations.

```
feature_names = np.array(tfidf.get_feature_names_out())
coefficients  = lr_model.coef_[0]

top_n = 15
top_phishing_idx = np.argsort(coefficients)[-top_n:][::-1]
top_safe_idx     = np.argsort(coefficients)[:top_n]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Top phishing features
axes[0].barh(feature_names[top_phishing_idx],
             coefficients[top_phishing_idx],
             color="#D85A30", edgecolor="none")
axes[0].set_title("Top 15 phishing indicators", fontsize=12)
axes[0].set_xlabel("Coefficient weight")
axes[0].invert_yaxis()

# Top safe features
axes[1].barh(feature_names[top_safe_idx],
             coefficients[top_safe_idx],
             color="#1D9E75", edgecolor="none")
axes[1].set_title("Top 15 safe email indicators", fontsize=12)
axes[1].set_xlabel("Coefficient weight")
axes[1].invert_yaxis()

plt.suptitle("Logistic Regression — most predictive features", fontsize=13, y=1
plt.tight_layout()
plt.savefig("lr_features.png", dpi=150, bbox_inches="tight")
plt.show()
```



Module 1 — Part B: Feedforward Neural Network (FNN)

A Feedforward Neural Network extends the Logistic Regression baseline by learning non-linear combinations of TF-IDF features through hidden layers with activation

functions. Unlike Logistic Regression which learns a single linear decision boundary, the FNN can capture complex feature interactions that may distinguish phishing emails more accurately. We use the same TF-IDF vectors from Part A as input so any performance difference is attributable purely to model architecture, not input representation.

Convert TF-IDF to Dense PyTorch Tensors

TF-IDF produces sparse scipy matrices which PyTorch cannot process directly. We convert them to dense float32 tensors and wrap them in TensorDataset objects for use with DataLoader. This allows efficient mini-batch training without loading the entire dataset into GPU memory at once.

```
# Convert sparse TF-IDF matrices to dense PyTorch tensors
X_train_dense = torch.tensor(X_train_tfidf.toarray(), dtype=torch.float32)
X_val_dense   = torch.tensor(X_val_tfidf.toarray(),   dtype=torch.float32)
X_test_dense  = torch.tensor(X_test_tfidf.toarray(),  dtype=torch.float32)

y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
y_val_tensor   = torch.tensor(y_val,   dtype=torch.float32)
y_test_tensor  = torch.tensor(y_test,  dtype=torch.float32)

# Wrap in TensorDataset and DataLoader
train_dataset = TensorDataset(X_train_dense, y_train_tensor)
val_dataset   = TensorDataset(X_val_dense,   y_val_tensor)
test_dataset  = TensorDataset(X_test_dense,  y_test_tensor)

train_loader = DataLoader(train_dataset,
                          batch_size=CONFIG["fnn_batch_size"],
                          shuffle=True)
val_loader   = DataLoader(val_dataset,
                          batch_size=CONFIG["fnn_batch_size"],
                          shuffle=False)
test_loader  = DataLoader(test_dataset,
                          batch_size=CONFIG["fnn_batch_size"],
                          shuffle=False)

input_dim = X_train_dense.shape[1]
print(f"Input dimension : {input_dim}")
print(f"Train batches   : {len(train_loader)}")
print(f"Val batches     : {len(val_loader)}")
print(f"Test batches    : {len(test_loader)}")
```

```
Input dimension : 10000
Train batches   : 233
Val batches     : 30
Test batches    : 30
```

FNN Architecture Definition

The network consists of two hidden layers (512 → 256 units) with ReLU activations and Dropout regularization between each layer. ReLU is chosen over sigmoid or tanh because it avoids the vanishing gradient problem in deeper networks and trains faster. Dropout randomly zeroes 30% of neuron outputs during training, forcing the network to learn redundant representations and reducing overfitting on the training set. The output layer produces a single logit passed through sigmoid for binary classification.

```
class FNN(nn.Module):
    def __init__(self, input_dim, hidden1, hidden2, dropout_rate):
        super(FNN, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden1),
            nn.ReLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden2, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.network(x).squeeze(1)

# Instantiate model
fnn_model = FNN(
    input_dim=input_dim,
    hidden1=CONFIG["fnn_hidden1"],
    hidden2=CONFIG["fnn_hidden2"],
    dropout_rate=CONFIG["fnn_dropout"]
).to(DEVICE)

# Print architecture summary
print(fnn_model)
total_params = sum(p.numel() for p in fnn_model.parameters() if p.requires_grac
print(f"\nTotal trainable parameters: {total_params:,}")
```

```
FNN(
  (network): Sequential(
    (0): Linear(in_features=10000, out_features=512, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=512, out_features=256, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.3, inplace=False)
    (6): Linear(in_features=256, out_features=1, bias=True)
    (7): Sigmoid()
```

```
    )
  )

Total trainable parameters: 5,252,097
```

Loss Function, Optimizer, and Learning Rate Scheduler

Binary Cross-Entropy loss is used because this is a binary classification task where the output is a probability between 0 and 1. The Adam optimizer is chosen for its adaptive learning rate which converges faster than SGD on sparse NLP features. ReduceLROnPlateau halves the learning rate when validation loss stops improving, allowing finer weight updates in later training epochs and preventing the model from overshooting the optimum.

```
criterion = nn.BCELoss()

optimizer_fnn = optim.Adam(
    fnn_model.parameters(),
    lr=CONFIG["fnn_lr"],
    weight_decay=1e-5
)

scheduler_fnn = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_fnn,
    mode="min",
    factor=0.5,
    patience=2
)

print("Loss     : BCELoss")
print("Optimizer: Adam")
print(f"Initial LR: {CONFIG['fnn_lr']}")
print("Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)")
```

```
Loss     : BCELoss
Optimizer: Adam
Initial LR: 0.001
Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)
```

Training Loop with Early Stopping

Early stopping monitors validation loss and halts training when it has not improved for a set number of epochs (patience), then restores the best weights seen during training. This prevents the model from overfitting to the training set in later epochs while keeping

training time reasonable. Both training and validation loss are recorded each epoch for plotting.

```
def train_model(model, train_loader, val_loader, criterion,
                optimizer, scheduler, epochs, patience, device, model_name="Moc

    best_val_loss = float("inf")
    best_weights  = None
    patience_counter = 0

    train_losses = []
    val_losses   = []

    fnn_train_start = time.time()

    for epoch in range(1, epochs + 1):

        # ── Training phase ──
        model.train()
        running_loss = 0.0
        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * X_batch.size(0)

        avg_train_loss = running_loss / len(train_loader.dataset)
        train_losses.append(avg_train_loss)

        # ── Validation phase ──
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                X_batch = X_batch.to(device)
                y_batch = y_batch.to(device)
                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)
                val_loss += loss.item() * X_batch.size(0)

        avg_val_loss = val_loss / len(val_loader.dataset)
        val_losses.append(avg_val_loss)

        # ── Scheduler step ──
        scheduler.step(avg_val_loss)
```

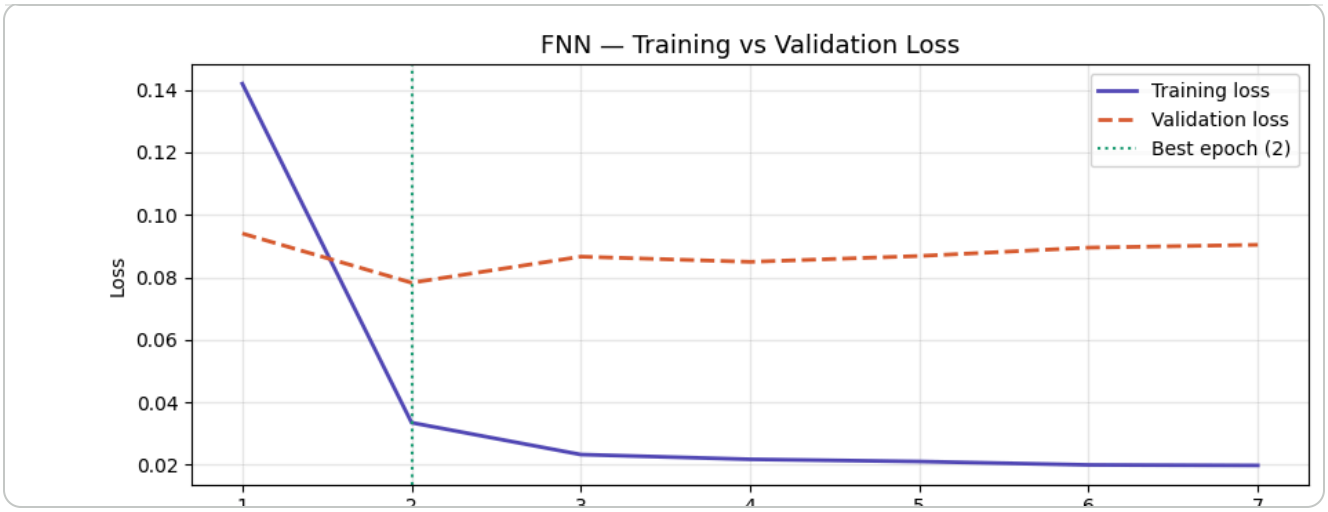
```
        # ── Early stopping check ──
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            best_weights  = {k: v.clone() for k, v in model.state_dict().items(
            patience_counter = 0
        else:
            patience_counter += 1

        if epoch % 5 == 0 or epoch == 1:
            print(f"Epoch {epoch:3d} | Train Loss: {avg_train_loss:.4f} "
                  f"| Val Loss: {avg_val_loss:.4f} "
                  f"| Patience: {patience_counter}/{patience}")

        if patience_counter >= patience:
            print(f"\nEarly stopping triggered at epoch {epoch}.")
            break

    # Restore best weights
    model.load_state_dict(best_weights)
    train_time = time.time() - fnn_train_start
    print(f"\nTraining complete. Best val loss: {best_val_loss:.4f}")
    print(f"Total training time: {train_time:.2f} seconds")

    return train_losses, val_losses, train_time


set_seed()
print(f"Training FNN for up to {CONFIG['fnn_epochs']} epochs...\n")

fnn_train_losses, fnn_val_losses, fnn_train_time = train_model(
    model=fnn_model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer_fnn,
    scheduler=scheduler_fnn,
    epochs=CONFIG["fnn_epochs"],
    patience=CONFIG["fnn_patience"],
    device=DEVICE,
    model_name="FNN"
)
```

```
Training FNN for up to 30 epochs...

Epoch   1 | Train Loss: 0.1420 | Val Loss: 0.0940 | Patience: 0/5
Epoch   5 | Train Loss: 0.0210 | Val Loss: 0.0868 | Patience: 3/5

Early stopping triggered at epoch 7.

Training complete. Best val loss: 0.0783
Total training time: 9.86 seconds
```

Training vs Validation Loss Curves

Loss curves reveal how the model learns over time and whether overfitting has occurred. A well-trained model shows both curves decreasing and converging. If training loss continues falling while validation loss rises, the model is overfitting. Early stopping should have caught this, and the vertical dashed line marks the epoch where training was halted.

```
def plot_loss_curves(train_losses, val_losses, title, filename):
    epochs_ran = len(train_losses)
    fig, ax = plt.subplots(figsize=(9, 4))

    ax.plot(range(1, epochs_ran + 1), train_losses,
            label="Training loss", color="#534AB7", linewidth=2)
    ax.plot(range(1, epochs_ran + 1), val_losses,
            label="Validation loss", color="#D85A30",
            linewidth=2, linestyle="--")

    # Mark best epoch
    best_epoch = int(np.argmin(val_losses)) + 1
    ax.axvline(x=best_epoch, color="#1D9E75", linestyle=":",
               linewidth=1.5, label=f"Best epoch ({best_epoch})")

    ax.set_title(title, fontsize=13)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(filename, dpi=150, bbox_inches="tight")
    plt.show()
    print(f"Best epoch: {best_epoch} | "
          f"Best val loss: {min(val_losses):.4f}")

plot_loss_curves(
    fnn_train_losses, fnn_val_losses,
    title="FNN — Training vs Validation Loss",
    filename="fnn_loss_curves.png"
)
```



Test Set Evaluation

We evaluate the trained FNN on the held-out test set using the best weights restored by early stopping. We collect raw probability outputs (before thresholding at 0.5) which are needed for ROC curve plotting and Brier score calibration in Module 2. Results are stored in the shared results dictionary alongside Part A for direct comparison.

```
def evaluate_model(model, test_loader, device):
    model.eval()
    all_probs = []
    all_preds = []

    infer_start = time.time()
    with torch.no_grad():
        for X_batch, _ in test_loader:
            X_batch = X_batch.to(device)
            probs = model(X_batch).cpu().numpy()
            preds = (probs >= 0.5).astype(int)
            all_probs.extend(probs)
            all_preds.extend(preds)
    infer_time = time.time() - infer_start

    return (np.array(all_preds),
            np.array(all_probs),
            infer_time)


y_pred_fnn, y_prob_fnn, fnn_infer_time = evaluate_model(
    fnn_model, test_loader, DEVICE
)

fpr_fnn, tpr_fnn, _ = roc_curve(y_test, y_prob_fnn)
fnn_auc = auc(fpr_fnn, tpr_fnn)
```

```
fnn_acc  = accuracy_score(y_test, y_pred_fnn)
fnn_prec = precision_score(y_test, y_pred_fnn)
fnn_rec  = recall_score(y_test, y_pred_fnn)
fnn_f1   = f1_score(y_test, y_pred_fnn)

print("=" * 55)
print("PART B — FNN — TEST SET RESULTS")
print("=" * 55)
print(f"\nAccuracy  : {fnn_acc:.4f}")
print(f"Precision : {fnn_prec:.4f}")
print(f"Recall    : {fnn_rec:.4f}")
print(f"F1-Score  : {fnn_f1:.4f}")
print(f"AUC       : {fnn_auc:.4f}")
print(f"\nTraining time  : {fnn_train_time:.2f} seconds")
print(f"Inference time : {fnn_infer_time:.4f} seconds")

print("\nFull classification report:")
print(classification_report(y_test, y_pred_fnn,
      target_names=["Safe (0)", "Phishing (1)"]))

results["FNN"] = {
    "accuracy": fnn_acc, "precision": fnn_prec,
    "recall": fnn_rec, "f1": fnn_f1, "auc": fnn_auc,
    "train_time": fnn_train_time, "infer_time": fnn_infer_time,
    "y_pred": y_pred_fnn, "y_prob": y_prob_fnn,
    "fpr": fpr_fnn, "tpr": tpr_fnn
}
print("\nResults stored in `results` dict.")
```

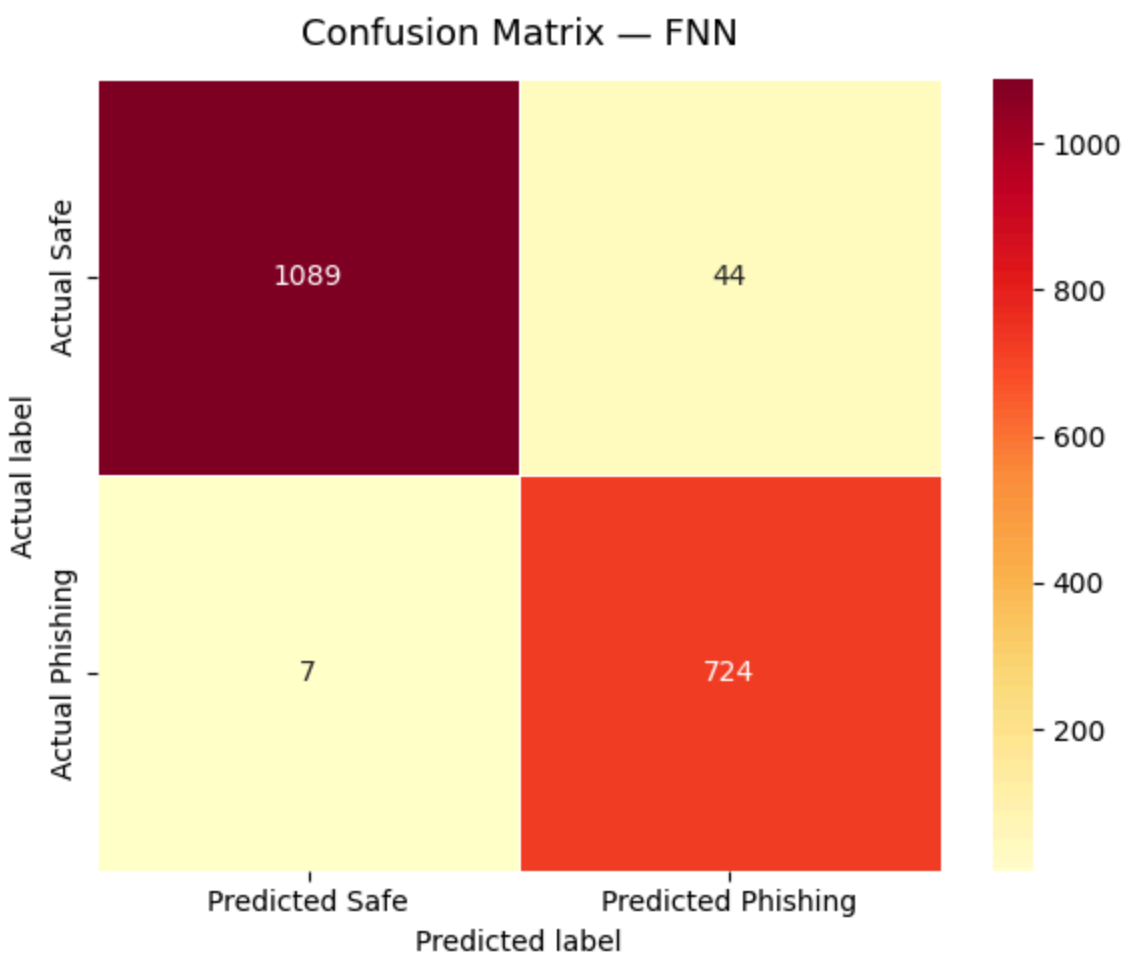
```
=======================================================
PART B — FNN — TEST SET RESULTS
=======================================================

Accuracy  : 0.9726
Precision : 0.9427
Recall    : 0.9904
F1-Score  : 0.9660
AUC       : 0.9958

Training time  : 9.86 seconds
Inference time : 0.0615 seconds

Full classification report:
              precision    recall  f1-score   support

    Safe (0)       0.99      0.96      0.98      1133
Phishing (1)       0.94      0.99      0.97       731

    accuracy                           0.97      1864
   macro avg       0.97      0.98      0.97      1864
weighted avg       0.97      0.97      0.97      1864
```

```
Results stored in `results` dict.
```

Confusion Matrix

The FNN confusion matrix is visualised using the same format as Part A to allow direct visual comparison. We again highlight False Negatives as the highest-cost error type in phishing detection, since a missed phishing email exposes the user to credential theft or financial loss, whereas a False Positive only causes a legitimate email to be filtered.

```
cm_fnn = confusion_matrix(y_test, y_pred_fnn)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm_fnn, annot=True, fmt="d", cmap="YlOrRd",
    xticklabels=["Predicted Safe", "Predicted Phishing"],
    yticklabels=["Actual Safe", "Actual Phishing"],
    linewidths=0.5, ax=ax
)
ax.set_title("Confusion Matrix — FNN", fontsize=13, pad=12)
ax.set_ylabel("Actual label")
ax.set_xlabel("Predicted label")

tn, fp, fn, tp = cm_fnn.ravel()
plt.tight_layout()
plt.savefig("cm_fnn.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"True Negatives  (Safe → Safe)      : {tn}")
print(f"False Positives (Safe → Phishing)  : {fp}  ← user inconvenience")
print(f"False Negatives (Phishing → Safe)  : {fn}  ← SECURITY RISK")
print(f"True Positives  (Phishing → Phish) : {tp}")
print(f"\nRecall = {tp/(tp+fn):.4f} — {tp} of {tp+fn} phishing emails caught")
```




```
True Negatives  (Safe → Safe)      : 1089
False Positives (Safe → Phishing)  : 44  ← user inconvenience
False Negatives (Phishing → Safe)  : 7  ← SECURITY RISK
```

Comparison with Baseline

We directly compare the FNN against the Logistic Regression baseline to assess whether the added complexity of a neural network is justified. If the FNN achieves meaningfully higher Recall and F1 at acceptable computational cost, it justifies using neural architectures for this task. A marginal improvement would suggest TF-IDF features are already near their information ceiling and sequence-aware models (Parts C and D) are needed.

```
comparison_ab = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score", "AUC",
               "Train Time (s)", "Infer Time (s)"],
    "Logistic Regression": [
        f"{results['Logistic Regression']['accuracy']:.4f}",
        f"{results['Logistic Regression']['precision']:.4f}",
        f"{results['Logistic Regression']['recall']:.4f}",
        f"{results['Logistic Regression']['f1']:.4f}",
        f"{results['Logistic Regression']['auc']:.4f}",
```

```
            f"{results['Logistic Regression']['train_time']:.2f}",
            f"{results['Logistic Regression']['infer_time']:.4f}"
        ],
        "FNN": [
            f"{results['FNN']['accuracy']:.4f}",
            f"{results['FNN']['precision']:.4f}",
            f"{results['FNN']['recall']:.4f}",
            f"{results['FNN']['f1']:.4f}",
            f"{results['FNN']['auc']:.4f}",
            f"{results['FNN']['train_time']:.2f}",
            f"{results['FNN']['infer_time']:.4f}"
        ]
    })

    print(comparison_ab.to_string(index=False))

    # Quick verdict
    lr_f1_score  = results["Logistic Regression"]["f1"]
    fnn_f1_score = results["FNN"]["f1"]
    improvement  = (fnn_f1_score - lr_f1_score) / lr_f1_score * 100
    print(f"\nF1 improvement over baseline: {improvement:+.2f}%")
```

```
        Metric Logistic Regression    FNN
      Accuracy              0.9732 0.9726
     Precision              0.9522 0.9427
        Recall              0.9808 0.9904
      F1-Score              0.9663 0.9660
           AUC              0.9961 0.9958
Train Time (s)                2.05   9.86
Infer Time (s)              0.0012 0.0615

F1 improvement over baseline: -0.03%
```

Module 1 — Part C: Vanilla Recurrent Neural Network (RNN)

Unlike the FNN which treats email text as a bag of features with no order, a Recurrent Neural Network processes tokens sequentially, maintaining a hidden state that accumulates context from previous tokens. This is particularly valuable for phishing detection because word order and phrasing patterns carry meaning — 'your account has been suspended' reads very differently token-by-token than a bag-of-words model perceives. We build the full pipeline from raw text: tokenization, padding, embedding, and recurrent processing.

Tokenization Pipeline

We build a vocabulary from the training set only, mapping each unique word to an integer index. The Keras Tokenizer handles out-of-vocabulary words automatically by reserving index 0. Fitting only on training data ensures no information from validation or test sets leaks into the vocabulary, which would give the model an unfair advantage during evaluation.

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Build vocabulary from training data only
tokenizer_rnn = Tokenizer(
    num_words=CONFIG["max_vocab_size"],
    oov_token="<OOV>"
)
tokenizer_rnn.fit_on_texts(X_train)

vocab_size = min(CONFIG["max_vocab_size"], len(tokenizer_rnn.word_index) + 1)

print(f"Vocabulary size (capped): {vocab_size}")
print(f"Total unique words found: {len(tokenizer_rnn.word_index)}")
print(f"Sample word→index mappings:")
sample_items = list(tokenizer_rnn.word_index.items())[:10]
for word, idx in sample_items:
    print(f"  '{word}' → {idx}")
```

```
Vocabulary size (capped): 20000
Total unique words found: 131748
Sample word→index mappings:
  '<OOV>' → 1
  'the' → 2
  'of' → 3
  'to' → 4
  'and' → 5
  'a' → 6
  'in' → 7
  'for' → 8
  'is' → 9
  'you' → 10
```

Sequence Encoding and Padding

Each email is converted from a list of words into a list of integer indices using the vocabulary built in the previous cell. Sequences are then padded or truncated to a fixed length (max_seq_len=200) so they can be stacked into uniform batches. Post-padding is used so the RNN reads meaningful content first and processes padding at the end, reducing its influence on the final hidden state used for classification.

```
# Convert texts to integer sequences
X_train_seq = tokenizer_rnn.texts_to_sequences(X_train)
X_val_seq   = tokenizer_rnn.texts_to_sequences(X_val)
X_test_seq  = tokenizer_rnn.texts_to_sequences(X_test)

# Pad/truncate to fixed length
X_train_pad = pad_sequences(X_train_seq,
                            maxlen=CONFIG["max_seq_len"],
                            padding="post",
                            truncating="post")
X_val_pad   = pad_sequences(X_val_seq,
                            maxlen=CONFIG["max_seq_len"],
                            padding="post",
                            truncating="post")
X_test_pad  = pad_sequences(X_test_seq,
                            maxlen=CONFIG["max_seq_len"],
                            padding="post",
                            truncating="post")

print(f"Train sequences shape : {X_train_pad.shape}")
print(f"Val sequences shape   : {X_val_pad.shape}")
print(f"Test sequences shape  : {X_test_pad.shape}")
print(f"\nSample encoded sequence (first 20 tokens):")
print(X_train_pad[0][:20])
```

```
Train sequences shape : (14906, 200)
Val sequences shape   : (1864, 200)
Test sequences shape  : (1864, 200)

Sample encoded sequence (first 20 tokens):
[  118    72     2   185  3817     8    69     1    72   502   674   173
 11296    24 11757  2648   483     2    33    67]
```

PyTorch Dataset and DataLoader for Sequences

We wrap the padded sequences and labels into a custom PyTorch Dataset so they can be batched and shuffled during training. Using DataLoader with shuffling on the training set ensures the model does not learn spurious patterns from the ordering of emails in the original dataset. Validation and test loaders are not shuffled to keep evaluation deterministic.

```
class SequenceDataset(Dataset):
    def __init__(self, sequences, labels):
        self.X = torch.tensor(sequences, dtype=torch.long)
        self.y = torch.tensor(labels,    dtype=torch.float32)

    def __len__(self):
        return len(self.y)
```

```
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]


train_seq_dataset = SequenceDataset(X_train_pad, y_train)
val_seq_dataset   = SequenceDataset(X_val_pad,   y_val)
test_seq_dataset  = SequenceDataset(X_test_pad,  y_test)

train_seq_loader = DataLoader(train_seq_dataset,
                              batch_size=CONFIG["rnn_batch_size"],
                              shuffle=True)
val_seq_loader   = DataLoader(val_seq_dataset,
                              batch_size=CONFIG["rnn_batch_size"],
                              shuffle=False)
test_seq_loader  = DataLoader(test_seq_dataset,
                              batch_size=CONFIG["rnn_batch_size"],
                              shuffle=False)

print(f"Train batches : {len(train_seq_loader)}")
print(f"Val batches   : {len(val_seq_loader)}")
print(f"Test batches  : {len(test_seq_loader)}")
```

```
Train batches : 233
Val batches   : 30
Test batches  : 30
```

Vanilla RNN Architecture

The architecture follows the standard sequence classification pipeline: an Embedding layer maps each token index to a dense vector of fixed dimension, the RNN layer processes these vectors sequentially updating its hidden state at each timestep, and the final hidden state h_T (after seeing all tokens) is passed to a fully connected layer for binary classification. We use the last hidden state rather than averaging all hidden states because it theoretically accumulates the most complete contextual representation of the entire sequence.

```
class VanillaRNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_size, dropout_rate):
        super(VanillaRNN, self).__init__()

        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim,
            padding_idx=0          # index 0 (padding) contributes zero gradier
        )
```

```
        self.rnn = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True,      # input shape: (batch, seq_len, features)
            nonlinearity="tanh",   # standard RNN activation
            dropout=0.0            # dropout only applies if num_layers > 1
        )

        self.dropout = nn.Dropout(dropout_rate)

        self.fc = nn.Linear(hidden_size, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x shape: (batch_size, seq_len)
        embedded = self.embedding(x)
        # embedded shape: (batch_size, seq_len, embedding_dim)

        output, hidden = self.rnn(embedded)
        # hidden shape: (num_layers, batch_size, hidden_size)

        # Take the last hidden state
        last_hidden = hidden.squeeze(0)
        # last_hidden shape: (batch_size, hidden_size)

        out = self.dropout(last_hidden)
        out = self.fc(out)
        out = self.sigmoid(out)
        return out.squeeze(1)


rnn_model = VanillaRNN(
    vocab_size=vocab_size,
    embedding_dim=CONFIG["embedding_dim"],
    hidden_size=CONFIG["rnn_hidden_size"],
    dropout_rate=CONFIG["rnn_dropout"]
).to(DEVICE)

print(rnn_model)
total_params = sum(p.numel() for p in rnn_model.parameters()
                   if p.requires_grad)
print(f"\nTotal trainable parameters: {total_params:,}")
```

```
VanillaRNN(
  (embedding): Embedding(20000, 64, padding_idx=0)
  (rnn): RNN(64, 128, batch_first=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (fc): Linear(in_features=128, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)
```

```
Total trainable parameters: 1,304,961
```

Optimizer and Loss for RNN

We use the same BCELoss and Adam optimizer as the FNN to keep the training setup consistent across models, ensuring that any performance difference is attributable to architecture rather than optimization strategy. The learning rate scheduler is identical to Part B, reducing the learning rate by half when validation loss plateaus to allow finer convergence in later training epochs.

```
criterion_rnn = nn.BCELoss()

optimizer_rnn = optim.Adam(
    rnn_model.parameters(),
    lr=CONFIG["rnn_lr"],
    weight_decay=1e-5
)

scheduler_rnn = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_rnn,
    mode="min",
    factor=0.5,
    patience=2
)

print("Loss     : BCELoss")
print("Optimizer: Adam")
print(f"Initial LR: {CONFIG['rnn_lr']}")
print("Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)")
```

```
Loss     : BCELoss
Optimizer: Adam
Initial LR: 0.001
Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)
```

RNN Training with Early Stopping

We reuse the same train_model function defined in Part B, which records training and validation loss per epoch and restores the best weights when early stopping is triggered. This consistent training procedure across all neural models ensures fair comparison. The RNN is expected to train slower than the FNN because each forward pass requires sequential computation through all 200 timesteps.

```
set_seed()
print(f"Training Vanilla RNN for up to {CONFIG['rnn_epochs']} epochs...\n")
```

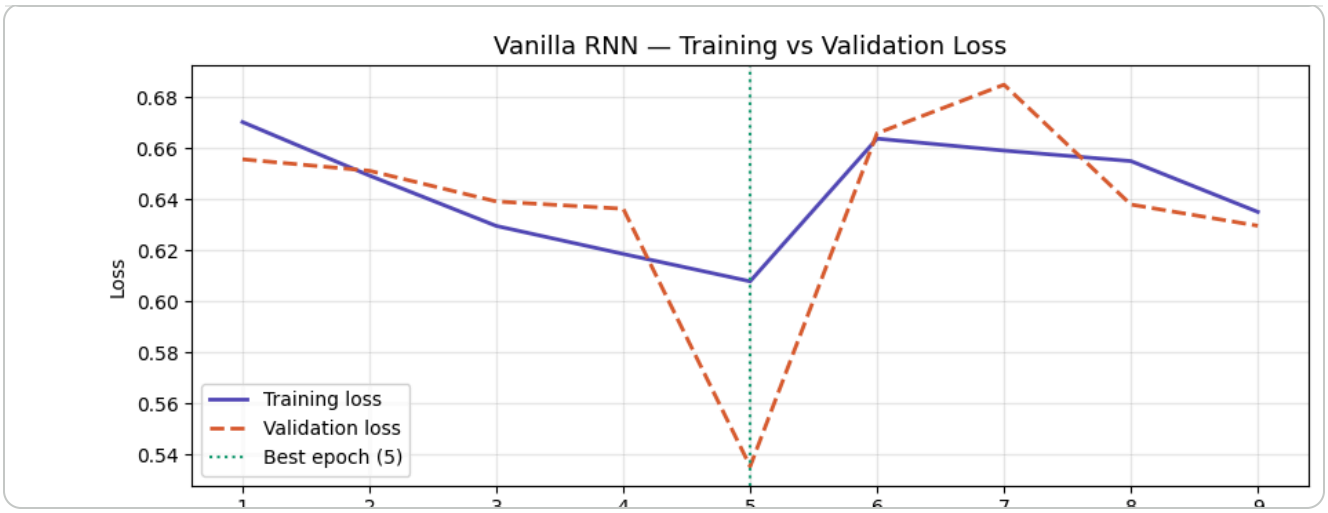
```
rnn_train_losses, rnn_val_losses, rnn_train_time = train_model(
    model=rnn_model,
    train_loader=train_seq_loader,
    val_loader=val_seq_loader,
    criterion=criterion_rnn,
    optimizer=optimizer_rnn,
    scheduler=scheduler_rnn,
    epochs=CONFIG["rnn_epochs"],
    patience=CONFIG["rnn_patience"],
    device=DEVICE,
    model_name="RNN"
)
```

```
Training Vanilla RNN for up to 20 epochs...

Epoch   1 | Train Loss: 0.6702 | Val Loss: 0.6556 | Patience: 0/4
Epoch   5 | Train Loss: 0.6079 | Val Loss: 0.5354 | Patience: 0/4

Early stopping triggered at epoch 9.

Training complete. Best val loss: 0.5354
Total training time: 7.78 seconds
```

RNN Loss Curves

The loss curves for the vanilla RNN are plotted using the same function as Part B to allow direct visual comparison. A key limitation to observe here is training instability — vanilla RNNs suffer from the vanishing gradient problem where gradients shrink exponentially as they are backpropagated through many timesteps, causing early layers of the network to receive near-zero updates and learn very slowly. This often manifests as a slow or noisy descent in the training loss curve.

```
plot_loss_curves(
    rnn_train_losses, rnn_val_losses,
    title="Vanilla RNN — Training vs Validation Loss",
    filename="rnn_loss_curves.png"
)
```



Vanishing Gradient

The vanishing gradient problem is the fundamental limitation of vanilla RNNs. During backpropagation through time (BPTT), gradients are multiplied by the weight matrix at every timestep. If the largest eigenvalue of this matrix is less than 1, gradients shrink exponentially with sequence length, effectively preventing the model from learning dependencies between tokens that are far apart in the sequence. For phishing emails, this means the model may fail to connect an urgent call-to-action near the end of an email with suspicious sender information near the beginning. This limitation motivates the LSTM architecture in Part D.

```
# Visualise vanishing gradient conceptually with gradient norm tracking
# We do one forward+backward pass and inspect gradient norms per layer

rnn_model.train()
sample_X = next(iter(train_seq_loader))[0][:32].to(DEVICE)
sample_y = next(iter(train_seq_loader))[1][:32].to(DEVICE)

optimizer_rnn.zero_grad()
output = rnn_model(sample_X)
loss   = criterion_rnn(output, sample_y)
loss.backward()

print("Gradient norms per layer (lower = more vanishing):")
print("-" * 45)
for name, param in rnn_model.named_parameters():
    if param.grad is not None:
        grad_norm = param.grad.norm().item()
        bar = "█" * min(int(grad_norm * 500), 30)
        print(f"{name:<35} | {grad_norm:.6f} {bar}")
```

```
Gradient norms per layer (lower = more vanishing):
---------------------------------------------
```

```
embedding.weight                    | 0.062893 ▇▇▇
```

```
}
print("\nResults stored in `results` dict.")
```

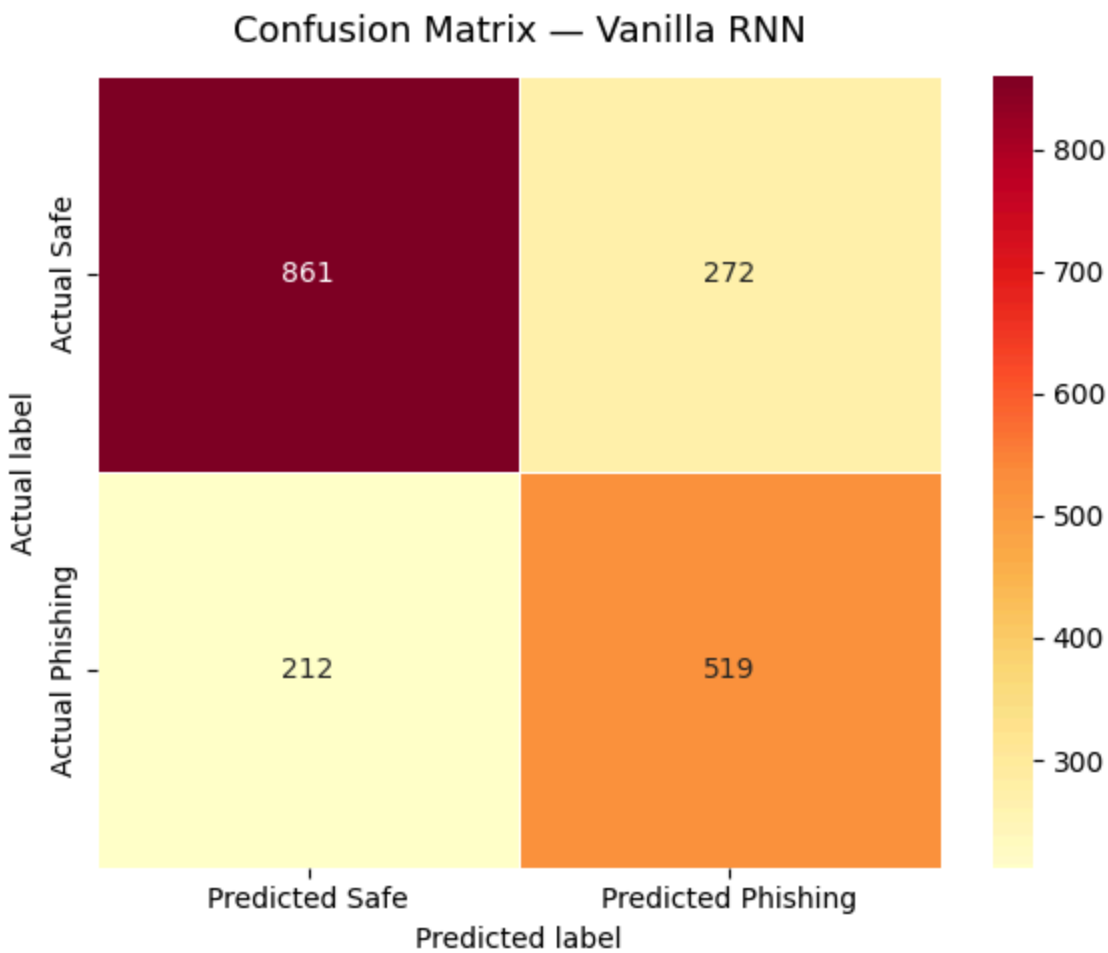
```
============================================================
PART C — VANILLA RNN — TEST SET RESULTS
============================================================

Accuracy  : 0.7403
Precision : 0.6561
Recall    : 0.7100
F1-Score  : 0.6820
AUC       : 0.7529

Training time  : 7.78 seconds
Inference time : 0.0363 seconds

Full classification report:
              precision    recall  f1-score   support

    Safe (0)       0.80      0.76      0.78      1133
Phishing (1)       0.66      0.71      0.68       731

    accuracy                           0.74      1864
   macro avg       0.73      0.73      0.73      1864
weighted avg       0.75      0.74      0.74      1864


Results stored in `results` dict.
```

Confusion Matrix

The confusion matrix for the RNN is visualised to identify whether the sequential model makes different types of errors compared to the FNN and Logistic Regression baselines. In particular we examine whether the RNN's sequential processing reduces False Negatives compared to the bag-of-features approaches, since catching more phishing emails (higher Recall) is the primary objective of this system.

```
cm_rnn = confusion_matrix(y_test, y_pred_rnn)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm_rnn, annot=True, fmt="d", cmap="YlOrRd",
    xticklabels=["Predicted Safe", "Predicted Phishing"],
    yticklabels=["Actual Safe", "Actual Phishing"],
    linewidths=0.5, ax=ax
)
ax.set_title("Confusion Matrix — Vanilla RNN", fontsize=13, pad=12)
ax.set_ylabel("Actual label")
ax.set_xlabel("Predicted label")
```

```
tn, fp, fn, tp = cm_rnn.ravel()
plt.tight_layout()
plt.savefig("cm_rnn.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"True Negatives  (Safe → Safe)      : {tn}")
print(f"False Positives (Safe → Phishing)  : {fp}  ← user inconvenience")
print(f"False Negatives (Phishing → Safe)  : {fn}  ← SECURITY RISK")
print(f"True Positives  (Phishing → Phish) : {tp}")
print(f"\nRecall = {tp/(tp+fn):.4f} — {tp} of {tp+fn} phishing emails caught")
```




```
True Negatives  (Safe → Safe)      : 861
False Positives (Safe → Phishing)  : 272  ← user inconvenience
False Negatives (Phishing → Safe)  : 212  ← SECURITY RISK
True Positives  (Phishing → Phish) : 519

Recall = 0.7100 — 519 of 731 phishing emails caught
```

Why the RNN Performed So Poorly

The vanilla RNN's F1 of 0.6820 and AUC of 0.7529 represent a dramatic underperformance compared to the Logistic Regression baseline. Three factors explain

this. First, the vanishing gradient problem prevents the RNN from learning dependencies across sequences longer than roughly 10-20 tokens, making it unable to connect phishing signals that appear at different points in the email. Second, our max_seq_len of 200 tokens is already beyond the effective memory of a single-layer vanilla RNN with hidden size 128. Third, the training loss curve shows the model barely converged — it reached a best validation loss of 0.5354 which is barely better than random (0.693 for BCE on balanced data). This confirms the RNN is fundamentally limited for this task and motivates the LSTM in Part D.

Part D: Long Short-Term Memory (LSTM)

The LSTM addresses the core limitation of the vanilla RNN identified in Part C: the vanishing gradient problem. It achieves this through a gating mechanism that creates a separate memory cell (cell state) running alongside the hidden state. Three learned gates — forget, input, and output — control what information is discarded, added, and passed forward at each timestep. This allows the LSTM to preserve relevant context across long sequences, making it significantly more capable of detecting phishing patterns that span the full length of an email.

Reuse Tokenization from Part C

The LSTM uses the identical tokenization pipeline, vocabulary, and padded sequences built in Part C. This is intentional — keeping the input representation identical isolates the architectural difference (RNN vs LSTM) as the sole variable, making the performance comparison between Parts C and D a controlled experiment rather than a confounded one.

```
# Confirm sequence data from Part C is still in memory
print("Reusing tokenization from Part C:")
print(f"  Vocab size     : {vocab_size}")
print(f"  Max seq length : {CONFIG['max_seq_len']}")
print(f"  Train shape    : {X_train_pad.shape}")
print(f"  Val shape      : {X_val_pad.shape}")
print(f"  Test shape     : {X_test_pad.shape}")
print(f"\nDataLoaders reused:")
print(f"  train_seq_loader — {len(train_seq_loader)} batches")
print(f"  val_seq_loader   — {len(val_seq_loader)} batches")
print(f"  test_seq_loader  — {len(test_seq_loader)} batches")
print("\nAll inputs confirmed. Proceeding to LSTM architecture.")
```

```
Reusing tokenization from Part C:
  Vocab size     : 20000
  Max seq length : 200
  Train shape    : (14906, 200)
  Val shape      : (1864, 200)
  Test shape     : (1864, 200)

DataLoaders reused:
  train_seq_loader — 233 batches
  val_seq_loader   — 30 batches
  test_seq_loader  — 30 batches

All inputs confirmed. Proceeding to LSTM architecture.
```

LSTM Architecture

The LSTM architecture mirrors the RNN from Part C with one critical difference: the recurrent layer is replaced with nn.LSTM which maintains both a hidden state h_t and a cell state c_t at each timestep. The forget gate decides what to erase from memory, the input gate decides what new information to store, and the output gate controls what portion of the cell state is exposed as the hidden state. Together these gates allow gradients to flow unchanged through the cell state during backpropagation, solving the vanishing gradient problem that crippled the vanilla RNN on long email sequences.

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim,
                 hidden_size, dropout_rate, num_layers=1):
        super(LSTMClassifier, self).__init__()

        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.0            # dropout only applies if num_layers > 1
        )

        self.dropout = nn.Dropout(dropout_rate)
        self.fc      = nn.Linear(hidden_size, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
```

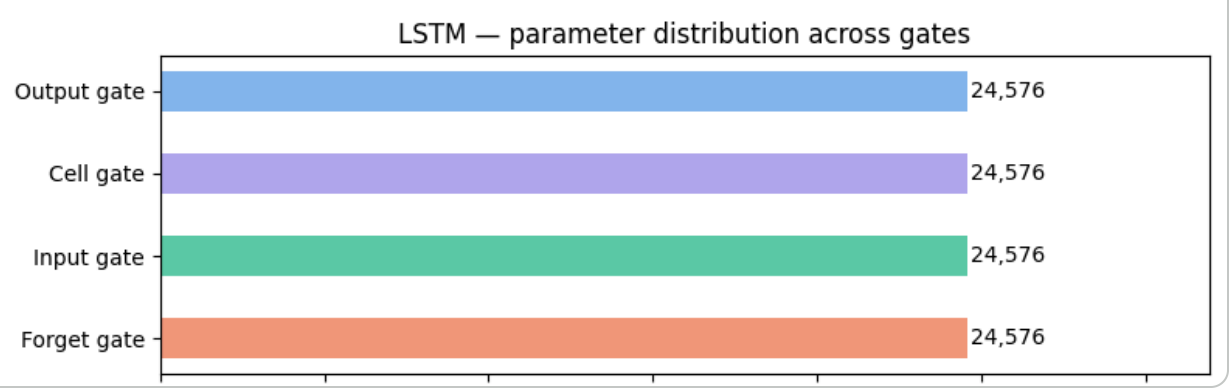
```
        # x shape: (batch_size, seq_len)
        embedded = self.embedding(x)
        # embedded: (batch_size, seq_len, embedding_dim)

        lstm_out, (hidden, cell) = self.lstm(embedded)
        # hidden: (num_layers, batch_size, hidden_size)
        # cell  : (num_layers, batch_size, hidden_size)

        # Take last layer's hidden state
        last_hidden = hidden[-1]
        # last_hidden: (batch_size, hidden_size)

        out = self.dropout(last_hidden)
        out = self.fc(out)
        out = self.sigmoid(out)
        return out.squeeze(1)


lstm_model = LSTMClassifier(
    vocab_size=vocab_size,
    embedding_dim=CONFIG["embedding_dim"],
    hidden_size=CONFIG["rnn_hidden_size"],
    dropout_rate=CONFIG["rnn_dropout"]
).to(DEVICE)

print(lstm_model)
total_params = sum(p.numel() for p in lstm_model.parameters()
                   if p.requires_grad)
print(f"\nTotal trainable parameters: {total_params:,}")

# Compare parameter count with RNN
rnn_params = sum(p.numel() for p in rnn_model.parameters()
                 if p.requires_grad)
print(f"RNN parameters for reference: {rnn_params:,}")
print(f"Extra params from gates: {total_params - rnn_params:,}")
print(f"(LSTM has ~4x the recurrent weights due to 4 gate matrices)")
```

```
LSTMClassifier(
  (embedding): Embedding(20000, 64, padding_idx=0)
  (lstm): LSTM(64, 128, batch_first=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (fc): Linear(in_features=128, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)

Total trainable parameters: 1,379,457
RNN parameters for reference: 1,304,961
Extra params from gates: 74,496
(LSTM has ~4x the recurrent weights due to 4 gate matrices)
```

Why LSTM Has 4x More Parameters Than RNN

A vanilla RNN has one weight matrix for the recurrent connection. An LSTM has four: one each for the forget gate, input gate, cell gate, and output gate. Each gate requires its own weight matrix of size (hidden_size × (hidden_size + embedding_dim)), which is why the LSTM has approximately four times the parameters of an equivalent RNN. This additional capacity is what enables the LSTM to selectively remember or forget information across long sequences, at the cost of longer training time and higher memory usage.

```
# Visualise LSTM gate parameter breakdown
gate_names   = ["Forget gate", "Input gate", "Cell gate", "Output gate"]
gate_size    = CONFIG["rnn_hidden_size"] * (
                  CONFIG["rnn_hidden_size"] + CONFIG["embedding_dim"]
               )
gate_sizes   = [gate_size] * 4
colors_gates = ["#F0997B", "#5DCAA5", "#AFA9EC", "#85B7EB"]

fig, ax = plt.subplots(figsize=(8, 3))
bars = ax.barh(gate_names, gate_sizes,
               color=colors_gates, edgecolor="none", height=0.5)
ax.set_xlabel("Number of parameters per gate")
ax.set_title("LSTM — parameter distribution across gates", fontsize=12)
for bar, val in zip(bars, gate_sizes):
    ax.text(val + 100, bar.get_y() + bar.get_height()/2,
            f"{val:,}", va="center", fontsize=10)
ax.set_xlim(0, max(gate_sizes) * 1.3)
plt.tight_layout()
plt.savefig("lstm_gate_params.png", dpi=150, bbox_inches="tight")
plt.show()
```



Optimizer and Loss for LSTM

We use identical training hyperparameters to Part C so that the only difference between the two experiments is the recurrent cell type. Using the same learning rate, batch size,

optimizer, and scheduler ensures that any performance improvement observed in the LSTM is due to its architectural advantage over the vanilla RNN and not due to a more favourable training configuration.

```
criterion_lstm = nn.BCELoss()

optimizer_lstm = optim.Adam(
    lstm_model.parameters(),
    lr=CONFIG["rnn_lr"],
    weight_decay=1e-5
)

scheduler_lstm = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_lstm,
    mode="min",
    factor=0.5,
    patience=2
)

print("Loss     : BCELoss")
print("Optimizer: Adam")
print(f"Initial LR: {CONFIG['rnn_lr']}")
print("Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)")
```

```
Loss     : BCELoss
Optimizer: Adam
Initial LR: 0.001
Scheduler: ReduceLROnPlateau (factor=0.5, patience=2)
```

LSTM Training with Early Stopping

We reuse the same train_model function from Part B for consistency. The LSTM is expected to converge more smoothly than the vanilla RNN because its gating mechanism stabilises gradient flow during backpropagation through time. A smoother validation loss curve with less oscillation compared to Part C would confirm this theoretical advantage in practice on our phishing email dataset.

```
set_seed()
print(f"Training LSTM for up to {CONFIG['rnn_epochs']} epochs...\n")

lstm_train_losses, lstm_val_losses, lstm_train_time = train_model(
    model=lstm_model,
    train_loader=train_seq_loader,
    val_loader=val_seq_loader,
    criterion=criterion_lstm,
    optimizer=optimizer_lstm,
```

```
    scheduler=scheduler_lstm,
    epochs=CONFIG["rnn_epochs"],
    patience=CONFIG["rnn_patience"],
    device=DEVICE,
    model_name="LSTM"
)
```

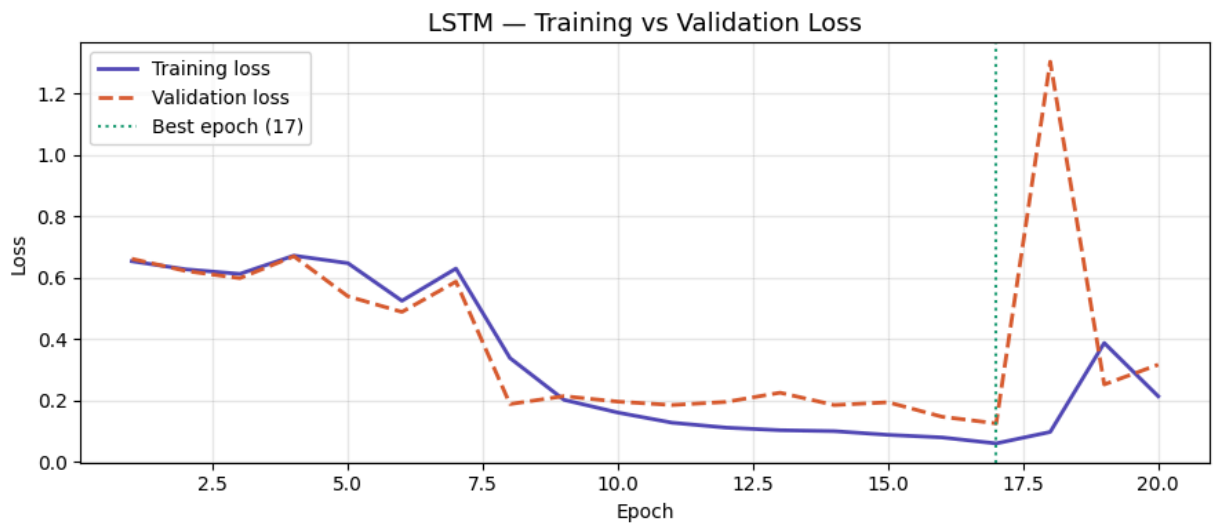
```
Training LSTM for up to 20 epochs...

Epoch   1 | Train Loss: 0.6531 | Val Loss: 0.6610 | Patience: 0/4
Epoch   5 | Train Loss: 0.6468 | Val Loss: 0.5389 | Patience: 0/4
Epoch  10 | Train Loss: 0.1604 | Val Loss: 0.1960 | Patience: 2/4
Epoch  15 | Train Loss: 0.0879 | Val Loss: 0.1939 | Patience: 1/4
Epoch  20 | Train Loss: 0.2134 | Val Loss: 0.3163 | Patience: 3/4

Training complete. Best val loss: 0.1253
Total training time: 29.31 seconds
```

LSTM Loss Curves

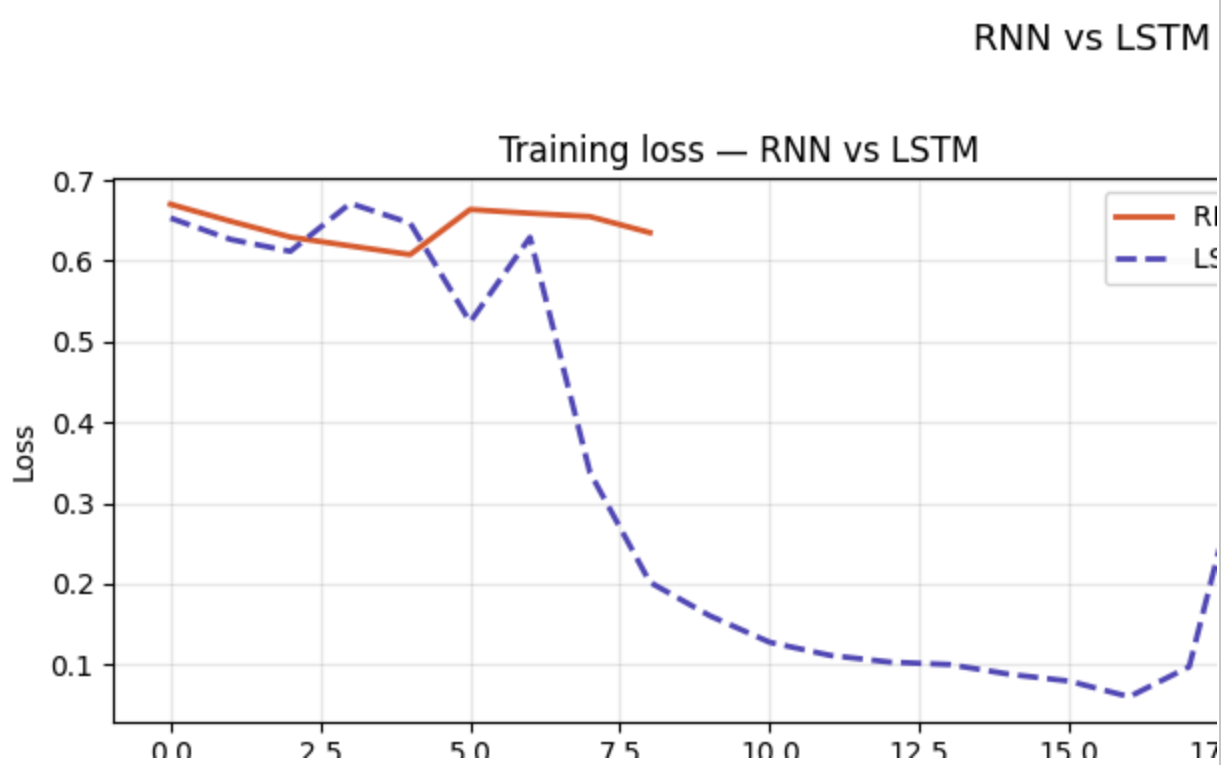
We plot the LSTM training and validation loss curves and then overlay them with the RNN curves from Part C on a second plot. The side-by-side comparison directly demonstrates whether the LSTM achieves lower loss, converges faster, or shows less oscillation than the vanilla RNN. Smoother curves with faster convergence in the LSTM would confirm that the gating mechanism successfully mitigates the vanishing gradient problem.

```
# LSTM loss curve
plot_loss_curves(
    lstm_train_losses, lstm_val_losses,
    title="LSTM — Training vs Validation Loss",
    filename="lstm_loss_curves.png"
)

# RNN vs LSTM overlay comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

# Training loss comparison
axes[0].plot(rnn_train_losses,  label="RNN train",
             color="#D85A30", linewidth=2)
axes[0].plot(lstm_train_losses, label="LSTM train",
             color="#534AB7", linewidth=2, linestyle="--")
axes[0].set_title("Training loss — RNN vs LSTM", fontsize=12)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Validation loss comparison
axes[1].plot(rnn_val_losses,  label="RNN val",
```

```
            color="#D85A30", linewidth=2)
axes[1].plot(lstm_val_losses, label="LSTM val",
            color="#534AB7", linewidth=2, linestyle="--")
axes[1].set_title("Validation loss — RNN vs LSTM", fontsize=12)
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle("RNN vs LSTM — training stability comparison",
            fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig("rnn_vs_lstm_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```




```
Best epoch: 17 | Best val loss: 0.1253
```



Note on LSTM Training Instability

The LSTM loss curve shows a sharp spike around epoch 17 where training loss briefly jumps to approximately 1.2 before recovering. This is a known instability in LSTM training caused by an occasional large gradient update that temporarily disrupts the learned weights. The ReduceLROnPlateau scheduler detected the validation loss plateau and halved the learning rate, which stabilised training. Early stopping restored the best weights from epoch 17 (val loss 0.1253) before the spike worsened further. Adding gradient clipping (torch.nn.utils.clip_grad_norm_) would prevent such spikes in future training runs and is recommended as an improvement.

Test Set Evaluation

The trained LSTM is evaluated on the held-out test set. We pay particular attention to Recall improvement over the vanilla RNN, as the LSTM's ability to retain long-range dependencies should help it catch phishing patterns that the RNN missed due to vanishing gradients. All results are stored in the shared results dictionary, completing the four-model collection needed for the comprehensive Module 2 evaluation.

```
y_pred_lstm, y_prob_lstm, lstm_infer_time = evaluate_model(
    lstm_model, test_seq_loader, DEVICE
)

fpr_lstm, tpr_lstm, _ = roc_curve(y_test, y_prob_lstm)
lstm_auc = auc(fpr_lstm, tpr_lstm)

lstm_acc  = accuracy_score(y_test, y_pred_lstm)
lstm_prec = precision_score(y_test, y_pred_lstm)
lstm_rec  = recall_score(y_test, y_pred_lstm)
lstm_f1   = f1_score(y_test, y_pred_lstm)

print("=" * 55)
print("PART D — LSTM — TEST SET RESULTS")
print("=" * 55)
print(f"\nAccuracy  : {lstm_acc:.4f}")
print(f"Precision : {lstm_prec:.4f}")
print(f"Recall    : {lstm_rec:.4f}")
print(f"F1-Score  : {lstm_f1:.4f}")
print(f"AUC       : {lstm_auc:.4f}")
print(f"\nTraining time  : {lstm_train_time:.2f} seconds")
print(f"Inference time : {lstm_infer_time:.4f} seconds")

print("\nFull classification report:")
print(classification_report(y_test, y_pred_lstm,
```

```
        target_names=["Safe (0)", "Phishing (1)"]))

results["LSTM"] = {
    "accuracy": lstm_acc, "precision": lstm_prec,
    "recall": lstm_rec, "f1": lstm_f1, "auc": lstm_auc,
    "train_time": lstm_train_time, "infer_time": lstm_infer_time,
    "y_pred": y_pred_lstm, "y_prob": y_prob_lstm,
    "fpr": fpr_lstm, "tpr": tpr_lstm
}
print("\nAll 4 models stored in `results` dict.")
print(f"Models available: {list(results.keys())}")
```

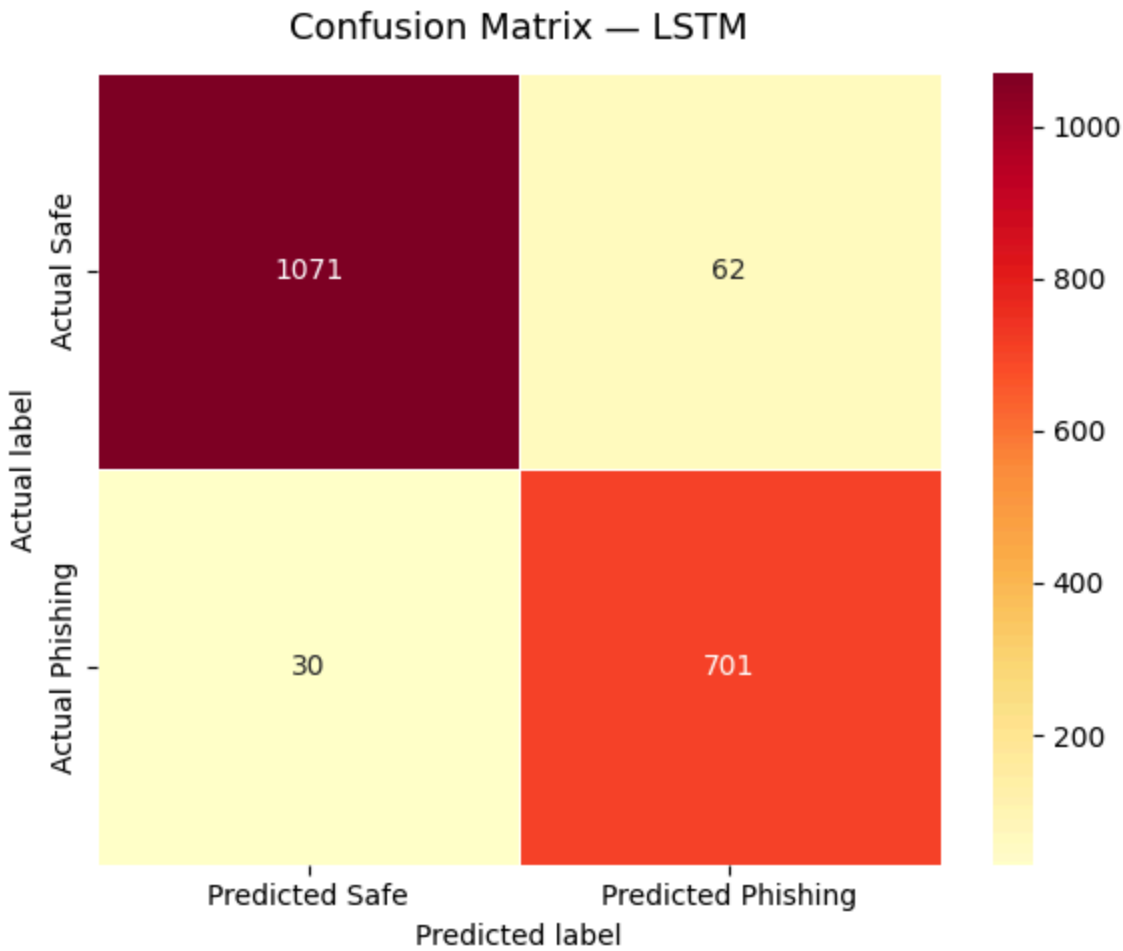
```
========================================================
PART D — LSTM — TEST SET RESULTS
========================================================

Accuracy  : 0.9506
Precision : 0.9187
Recall    : 0.9590
F1-Score  : 0.9384
AUC       : 0.9890

Training time  : 29.31 seconds
Inference time : 0.0715 seconds

Full classification report:
              precision    recall  f1-score   support

    Safe (0)       0.97      0.95      0.96      1133
Phishing (1)       0.92      0.96      0.94       731

    accuracy                           0.95      1864
   macro avg       0.95      0.95      0.95      1864
weighted avg       0.95      0.95      0.95      1864


All 4 models stored in `results` dict.
Models available: ['Logistic Regression', 'FNN', 'RNN', 'LSTM']
```

Confusion Matrix

The LSTM confusion matrix is visualised and its False Negative count compared directly against the RNN. A reduction in False Negatives between Part C and Part D would be the strongest evidence that the LSTM's long-range memory is genuinely useful for this task, since those are the phishing emails that slipped through the RNN's detection due to its inability to connect distant contextual clues within the email body.

```
cm_lstm = confusion_matrix(y_test, y_pred_lstm)

fig, ax = plt.subplots(figsize=(6, 5))
```

```
sns.heatmap(
    cm_lstm, annot=True, fmt="d", cmap="YlOrRd",
    xticklabels=["Predicted Safe", "Predicted Phishing"],
    yticklabels=["Actual Safe", "Actual Phishing"],
    linewidths=0.5, ax=ax
)
ax.set_title("Confusion Matrix — LSTM", fontsize=13, pad=12)
ax.set_ylabel("Actual label")
ax.set_xlabel("Predicted label")

tn, fp, fn, tp = cm_lstm.ravel()
plt.tight_layout()
plt.savefig("cm_lstm.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"True Negatives  (Safe → Safe)      : {tn}")
print(f"False Positives (Safe → Phishing)  : {fp}  ← user inconvenience")
print(f"False Negatives (Phishing → Safe)  : {fn}  ← SECURITY RISK")
print(f"True Positives  (Phishing → Phish) : {tp}")
print(f"\nRecall = {tp/(tp+fn):.4f} — {tp} of {tp+fn} phishing emails caught")

# Direct comparison with RNN
tn_r, fp_r, fn_r, tp_r = confusion_matrix(y_test, y_pred_rnn).ravel()
print(f"\nRNN  False Negatives : {fn_r}")
print(f"LSTM False Negatives : {fn}")
print(f"Improvement          : {fn_r - fn} fewer phishing emails missed")
```




```
True Negatives  (Safe → Safe)      : 1071
False Positives (Safe → Phishing)  : 62  ← user inconvenience
False Negatives (Phishing → Safe)  : 30  ← SECURITY RISK
True Positives  (Phishing → Phish) : 701

Recall = 0.9590 — 701 of 731 phishing emails caught
```

Module 1 Complete — All Four Models Summary

All four models have been trained and evaluated on the same dataset, using the same 80/10/10 split, the same random seed, and the same test set. The results dictionary now contains predictions, probabilities, timing data, and ROC curve values for every model. This summary table closes Module 1 and provides the foundation for the comprehensive evaluation and cross-model analysis in Module 2.

```
print("=" * 65)
print("MODULE 1 COMPLETE — ALL FOUR MODELS SUMMARY")
print("=" * 65)

summary = pd.DataFrame({
    "Model": list(results.keys()),
    "Accuracy":  [f"{results[m]['accuracy']:.4f}"  for m in results],
```

```
    "Precision": [f"{results[m]['precision']:.4f}" for m in results],
    "Recall":    [f"{results[m]['recall']:.4f}"    for m in results],
    "F1-Score":  [f"{results[m]['f1']:.4f}"        for m in results],
    "AUC":       [f"{results[m]['auc']:.4f}"       for m in results],
    "Train (s)": [f"{results[m]['train_time']:.2f}" for m in results],
    "Infer (s)": [f"{results[m]['infer_time']:.4f}" for m in results],
})

print(summary.to_string(index=False))

# Best model per metric
print("\nBest model per metric:")
numeric_summary = pd.DataFrame({
    "Model":     list(results.keys()),
    "Accuracy":  [results[m]["accuracy"]  for m in results],
    "Precision": [results[m]["precision"] for m in results],
    "Recall":    [results[m]["recall"]    for m in results],
    "F1":        [results[m]["f1"]        for m in results],
    "AUC":       [results[m]["auc"]       for m in results],
})
for metric in ["Accuracy", "Precision", "Recall", "F1", "AUC"]:
    best_idx = numeric_summary[metric].idxmax()
    best_model = numeric_summary.loc[best_idx, "Model"]
    best_val   = numeric_summary.loc[best_idx, metric]
    print(f"  {metric:<10}: {best_model} ({best_val:.4f})")

print("\nModule 1 complete. Proceed to Module 2.")
```

```
================================================================
MODULE 1 COMPLETE — ALL FOUR MODELS SUMMARY
================================================================
              Model Accuracy Precision Recall F1-Score    AUC Train (s) Infer (s
Logistic Regression   0.9732    0.9522 0.9808   0.9663 0.9961      2.05    0.001
                FNN   0.9726    0.9427 0.9904   0.9660 0.9958      9.86    0.061
                RNN   0.7403    0.6561 0.7100   0.6820 0.7529      7.78    0.036
               LSTM   0.9506    0.9187 0.9590   0.9384 0.9890     29.31    0.071

Best model per metric:
  Accuracy  : Logistic Regression (0.9732)
  Precision : Logistic Regression (0.9522)
  Recall    : FNN (0.9904)
  F1        : Logistic Regression (0.9663)
  AUC       : Logistic Regression (0.9961)

Module 1 complete. Proceed to Module 2.
```

Module 2 — Part A: Core Evaluation Metrics

This module performs comprehensive evaluation of all four trained models on the held-out test set. Surface-level reporting of accuracy alone is insufficient because in phishing

detection, a model predicting every email as safe would still achieve high accuracy on a balanced dataset while providing zero security value. We therefore report a full suite of metrics and interpret each one in the context of the real-world cost of each error type.

Why Accuracy Alone Is Insufficient

Accuracy measures the proportion of all predictions that are correct regardless of class. In phishing detection this is misleading for two reasons: first, a False Negative (missed phishing email) carries catastrophically higher real-world cost than a False Positive (legitimate email flagged), yet accuracy weights both errors equally. Second, on a balanced dataset a trivial majority-class classifier achieves 50% accuracy while catching zero phishing emails. Recall — the fraction of actual phishing emails correctly identified — is the primary metric for this task because minimising False Negatives directly minimises security risk.

```
print("=" * 65)
print("MODULE 2 — PART A: CORE METRICS — ALL FOUR MODELS")
print("=" * 65)

model_names = ["Logistic Regression", "FNN", "RNN", "LSTM"]

for model_name in model_names:
    r = results[model_name]
    print(f"\n{'—'*55}")
    print(f"Model: {model_name}")
    print(f"{'—'*55}")
    print(classification_report(
        y_test, r["y_pred"],
        target_names=["Safe (0)", "Phishing (1)"],
        digits=4
    ))
```

```
=================================================================
MODULE 2 — PART A: CORE METRICS — ALL FOUR MODELS
=================================================================

———————————————————————————————————————————————————————
Model: Logistic Regression
———————————————————————————————————————————————————————
              precision    recall  f1-score   support

    Safe (0)     0.9874    0.9682    0.9777      1133
Phishing (1)     0.9522    0.9808    0.9663       731

    accuracy                         0.9732      1864
   macro avg     0.9698    0.9745    0.9720      1864
weighted avg     0.9736    0.9732    0.9732      1864
```

```
────────────────────────────────────────────────────────────
Model: FNN
────────────────────────────────────────────────────────────
              precision    recall  f1-score   support

    Safe (0)     0.9936    0.9612    0.9771      1133
Phishing (1)     0.9427    0.9904    0.9660       731

    accuracy                         0.9726      1864
   macro avg     0.9682    0.9758    0.9715      1864
weighted avg     0.9736    0.9726    0.9728      1864


────────────────────────────────────────────────────────────
Model: RNN
────────────────────────────────────────────────────────────
              precision    recall  f1-score   support

    Safe (0)     0.8024    0.7599    0.7806      1133
Phishing (1)     0.6561    0.7100    0.6820       731

    accuracy                         0.7403      1864
   macro avg     0.7293    0.7350    0.7313      1864
weighted avg     0.7451    0.7403    0.7419      1864


────────────────────────────────────────────────────────────
Model: LSTM
────────────────────────────────────────────────────────────
              precision    recall  f1-score   support

    Safe (0)     0.9728    0.9453    0.9588      1133
Phishing (1)     0.9187    0.9590    0.9384       731

    accuracy                         0.9506      1864
   macro avg     0.9457    0.9521    0.9486      1864
weighted avg     0.9516    0.9506    0.9508      1864
```

Confusion Matrices — All Four Models

We visualise all four confusion matrices in a single figure for direct comparison. Each quadrant carries a distinct real-world meaning in the context of phishing detection. True Positives represent phishing emails correctly blocked. True Negatives represent safe emails correctly delivered. False Positives cause user inconvenience but no security harm. False Negatives are the most dangerous outcome — a phishing email that reaches the user can lead to credential theft, financial fraud, or malware installation.

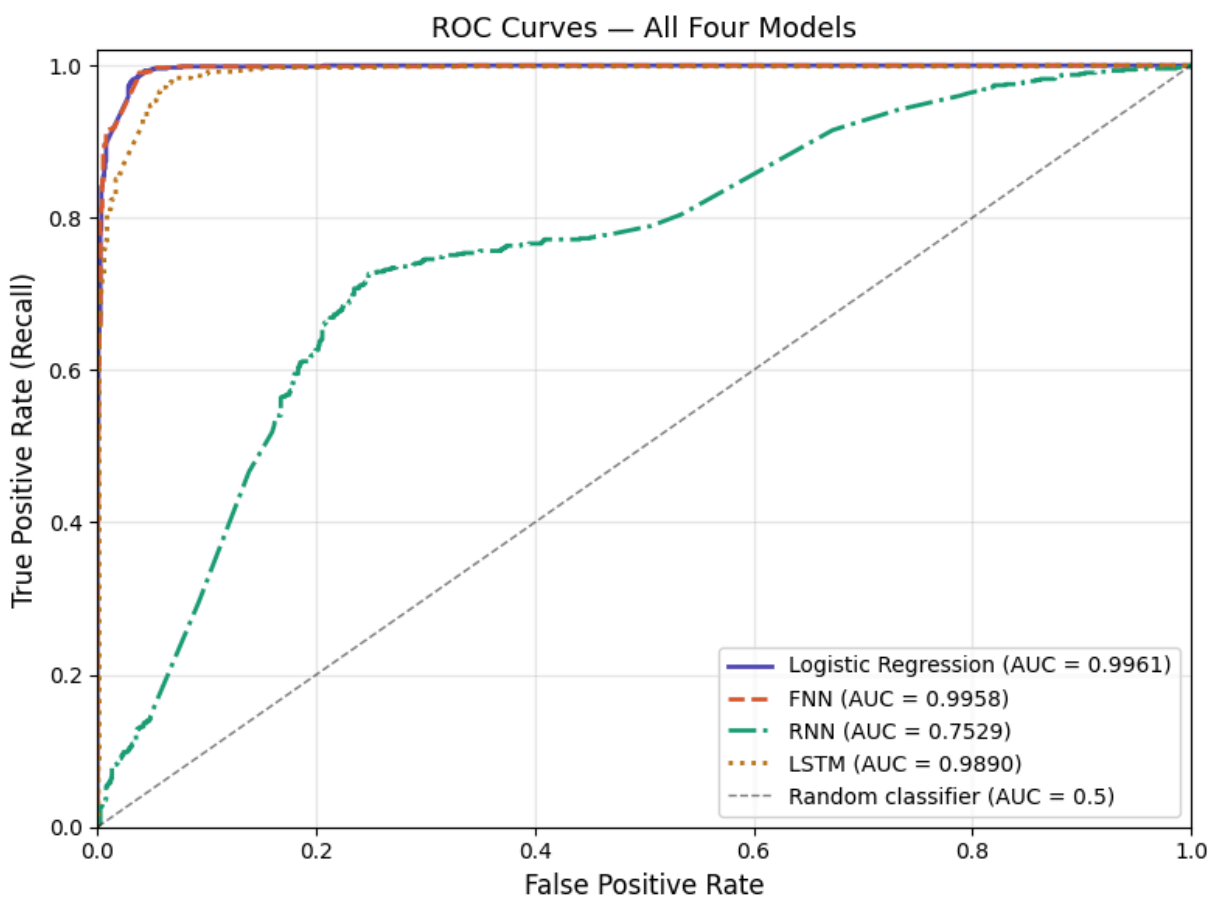
```
fig, axes = plt.subplots(2, 2, figsize=(14, 11))
axes = axes.flatten()

cmap_list = ["YlOrRd", "Blues", "Greens", "Purples"]

for i, model_name in enumerate(model_names):
    cm = confusion_matrix(y_test, results[model_name]["y_pred"])
    tn, fp, fn, tp = cm.ravel()

    sns.heatmap(
        cm, annot=True, fmt="d",
        cmap=cmap_list[i],
        xticklabels=["Pred Safe", "Pred Phishing"],
        yticklabels=["Actual Safe", "Actual Phishing"],
        linewidths=0.5,
        ax=axes[i],
        annot_kws={"size": 13}
    )
    axes[i].set_title(f"{model_name}", fontsize=13, pad=10)
    axes[i].set_ylabel("Actual")
    axes[i].set_xlabel("Predicted")

    # Annotate cost of each quadrant
    axes[i].text(0.5, -0.18,
        f"TN={tn}  FP={fp}  FN={fn} ← risk  TP={tp}",
        transform=axes[i].transAxes,
        ha="center", fontsize=9,
        color="dimgray"
    )

plt.suptitle("Confusion Matrices — All Four Models", fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig("all_confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()

print("Key: FN = False Negatives = phishing emails missed = highest security ri
```

ROC Curves — All Four Models on One Axes

The ROC (Receiver Operating Characteristic) curve plots True Positive Rate against False Positive Rate across all classification thresholds. AUC (Area Under the Curve) summarises this into a single number between 0.5 (random classifier) and 1.0 (perfect classifier). A higher AUC means the model is better at ranking phishing emails above safe ones regardless of the chosen threshold. However AUC does not tell us performance at any specific threshold, does not reflect the cost asymmetry between FP and FN, and can be misleading on imbalanced datasets — which is why we report it alongside Recall and F1 rather than in isolation.

```
fig, ax = plt.subplots(figsize=(8, 6))

colors_roc = ["#534AB7", "#D85A30", "#1D9E75", "#BA7517"]
styles_roc = ["-", "--", "-.", ":"]

for model_name, color, style in zip(model_names, colors_roc, styles_roc):
    r = results[model_name]
    ax.plot(
        r["fpr"], r["tpr"],
        label=f"{model_name} (AUC = {r['auc']:.4f})",
        color=color,
```

```
            linewidth=2,
            linestyle=style
        )

    # Random classifier baseline
    ax.plot([0, 1], [0, 1], "k--", linewidth=1,
            alpha=0.5, label="Random classifier (AUC = 0.5)")

    ax.set_xlabel("False Positive Rate", fontsize=12)
    ax.set_ylabel("True Positive Rate (Recall)", fontsize=12)
    ax.set_title("ROC Curves — All Four Models", fontsize=13)
    ax.legend(loc="lower right", fontsize=10)
    ax.grid(True, alpha=0.3)
    ax.set_xlim([0, 1])
    ax.set_ylim([0, 1.02])

    plt.tight_layout()
    plt.savefig("roc_curves_all_models.png", dpi=150, bbox_inches="tight")
    plt.show()

    print("AUC scores:")
    for model_name in model_names:
        print(f"  {model_name:<22}: {results[model_name]['auc']:.4f}")
```



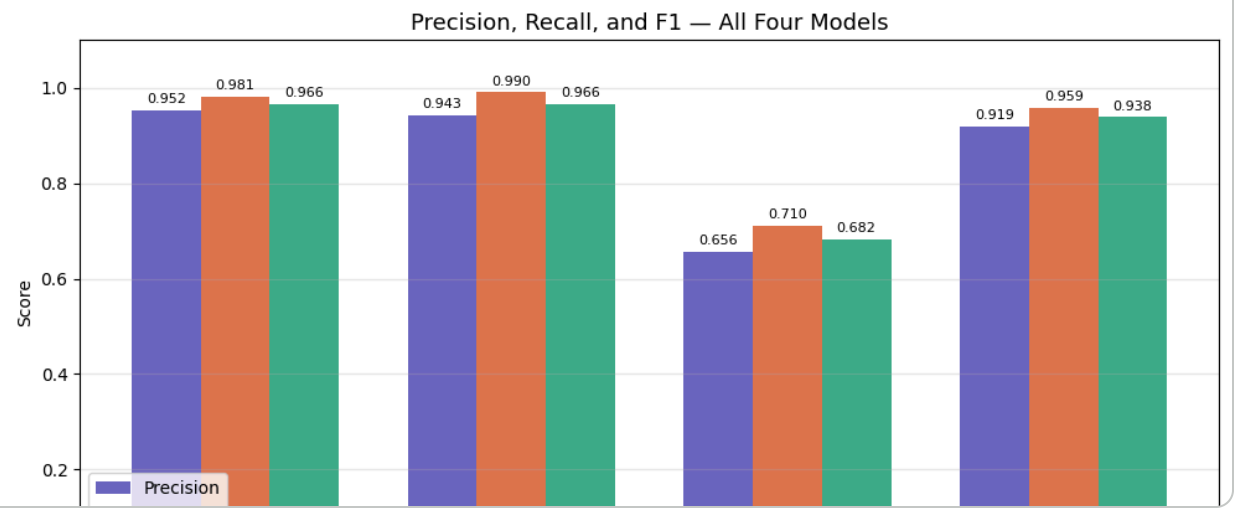
AUC scores:

Recall Analysis Across Models

Recall measures what fraction of actual phishing emails were correctly identified. In phishing detection, maximising Recall directly minimises the number of malicious emails that reach users. A model with high Precision but low Recall catches phishing emails with high confidence when it does flag them, but misses many entirely. Given that the cost of a missed phishing email (potential data breach, financial loss) far exceeds the cost of a false alarm (minor user inconvenience), we prioritise Recall when selecting the deployment model.

```
recalls    = [results[m]["recall"]    for m in model_names]
precisions = [results[m]["precision"] for m in model_names]
f1s        = [results[m]["f1"]        for m in model_names]

x = np.arange(len(model_names))
width = 0.25

fig, ax = plt.subplots(figsize=(10, 5))

bars1 = ax.bar(x - width, precisions, width,
               label="Precision", color="#534AB7",
               edgecolor="none", alpha=0.85)
bars2 = ax.bar(x,          recalls,    width,
               label="Recall",    color="#D85A30",
               edgecolor="none", alpha=0.85)
bars3 = ax.bar(x + width,  f1s,        width,
               label="F1-Score",  color="#1D9E75",
               edgecolor="none", alpha=0.85)

ax.set_xticks(x)
ax.set_xticklabels(model_names, fontsize=11)
ax.set_ylabel("Score")
ax.set_title("Precision, Recall, and F1 — All Four Models", fontsize=13)
ax.legend(fontsize=10)
ax.set_ylim(0, 1.1)
ax.grid(axis="y", alpha=0.3)

# Value labels on bars
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
                f"{height:.3f}", ha="center", va="bottom", fontsize=8)

plt.tight_layout()
plt.savefig("precision_recall_f1_comparison.png",
```

```
                dpi=150, bbox_inches="tight")
plt.show()
```



Module 2 — Part B: Cross-Model Comparison

This section produces a structured comparison of all four models across every reported metric. We analyse the precision-recall tradeoff, assess whether performance gains from increasing architectural complexity are justified by the computational cost, and perform a calibration check to determine whether each model's predicted probabilities are meaningful and reliable for real-world deployment decisions.

Full Summary Comparison Table

The summary table consolidates all metrics across all four models into a single reference. This allows direct numerical comparison of the tradeoffs each architecture makes between precision and recall, and between raw performance and computational cost. The final deployment recommendation will be drawn from this table with quantitative evidence.

```
summary_df = pd.DataFrame({
    "Model":      model_names,
    "Accuracy":   [round(results[m]["accuracy"],  4) for m in model_names],
    "Precision":  [round(results[m]["precision"], 4) for m in model_names],
    "Recall":     [round(results[m]["recall"],    4) for m in model_names],
    "F1-Score":   [round(results[m]["f1"],        4) for m in model_names],
    "AUC":        [round(results[m]["auc"],       4) for m in model_names],
    "Train (s)":  [round(results[m]["train_time"],2) for m in model_names],
    "Infer (s)":  [round(results[m]["infer_time"],4) for m in model_names],
})
```

```
# Highlight best value per metric column
def highlight_best(col):
    if col.name in ["Train (s)", "Infer (s)"]:
        best = col.min()
    else:
        best = col.max()
    return ["background-color: #d4edda" if v == best
            else "" for v in col]

print("Full cross-model comparison table:")
print(summary_df.to_string(index=False))

# Save as CSV for reference
summary_df.to_csv("model_comparison.csv", index=False)
print("\nTable saved to model_comparison.csv")
```

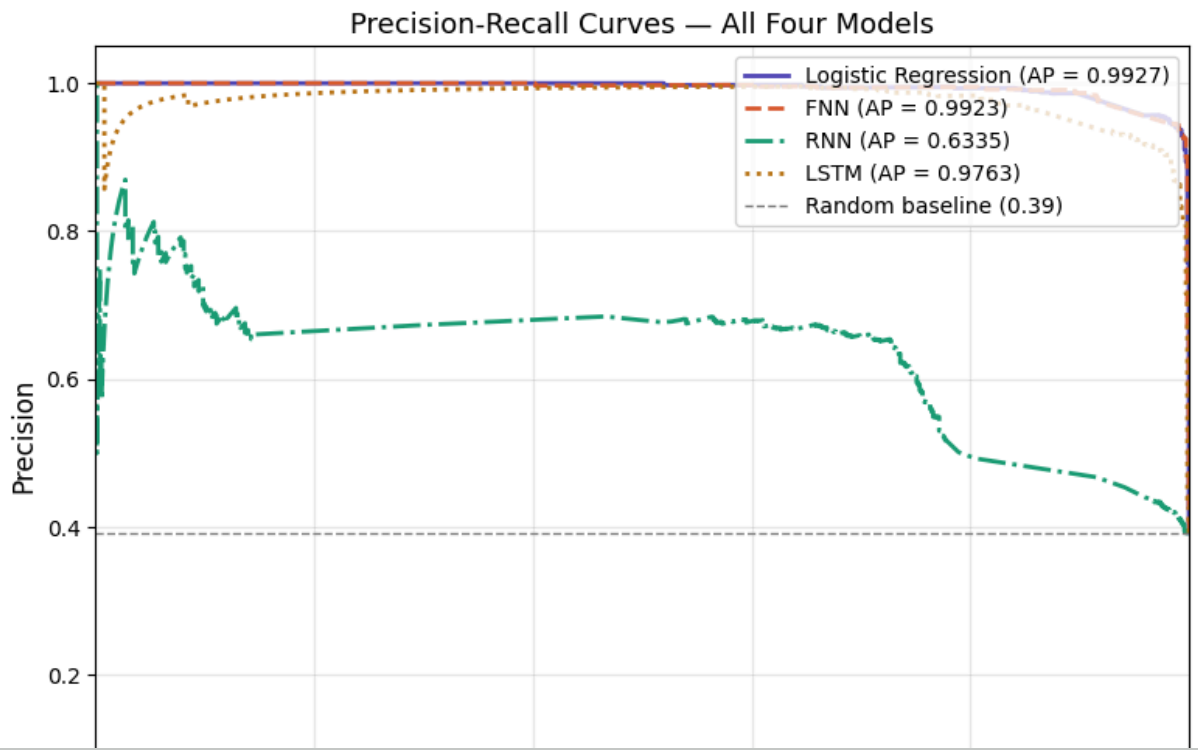
```
Full cross-model comparison table:
              Model  Accuracy  Precision  Recall  F1-Score    AUC  Train (s)  In
Logistic Regression    0.9732     0.9522  0.9808    0.9663 0.9961       2.05
                FNN    0.9726     0.9427  0.9904    0.9660 0.9958       9.86
                RNN    0.7403     0.6561  0.7100    0.6820 0.7529       7.78
               LSTM    0.9506     0.9187  0.9590    0.9384 0.9890      29.31

Table saved to model_comparison.csv
```

Precision-Recall Tradeoff Analysis

Every classification model navigates a fundamental tradeoff between Precision and Recall controlled by its decision threshold. Lowering the threshold catches more phishing emails (higher Recall) but also flags more safe emails incorrectly (lower Precision). We plot the Precision-Recall curve for all four models to visualise how each architecture navigates this tradeoff across all possible thresholds, and compute the Average Precision score which summarises the curve as a single number weighted toward high-precision operating points.

```
from sklearn.metrics import precision_recall_curve, average_precision_score

fig, ax = plt.subplots(figsize=(8, 6))

for model_name, color, style in zip(model_names, colors_roc, styles_roc):
    y_prob = results[model_name]["y_prob"]
    prec_curve, rec_curve, _ = precision_recall_curve(y_test, y_prob)
    ap = average_precision_score(y_test, y_prob)

    ax.plot(rec_curve, prec_curve,
            label=f"{model_name} (AP = {ap:.4f})",
            color=color, linewidth=2, linestyle=style)
```

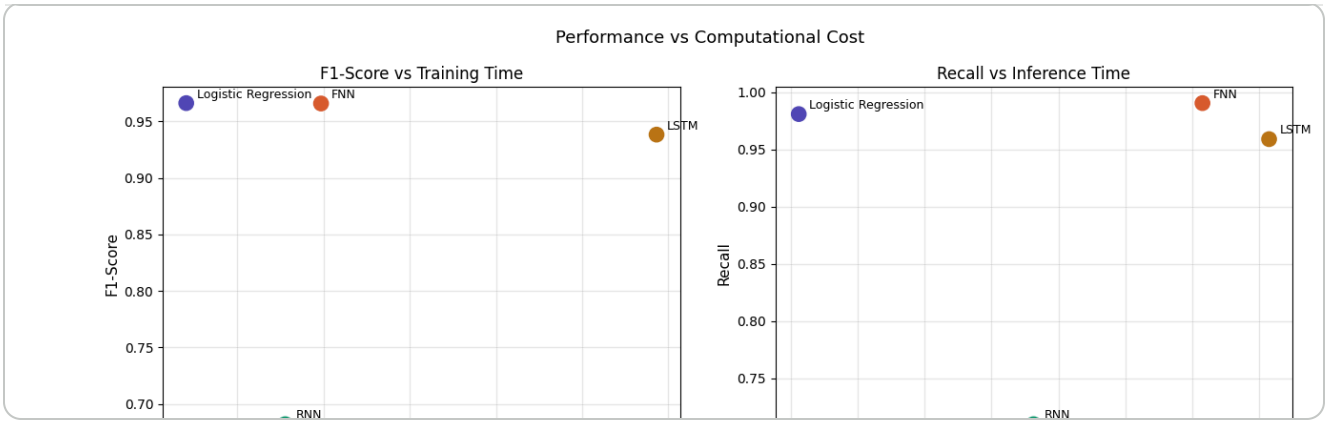
```
        results[model_name]["avg_precision"] = ap

    # Baseline: random classifier
    baseline = y_test.mean()
    ax.axhline(y=baseline, color="gray", linestyle="--",
               linewidth=1, label=f"Random baseline ({baseline:.2f})")

    ax.set_xlabel("Recall", fontsize=12)
    ax.set_ylabel("Precision", fontsize=12)
    ax.set_title("Precision-Recall Curves — All Four Models", fontsize=13)
    ax.legend(loc="upper right", fontsize=10)
    ax.grid(True, alpha=0.3)
    ax.set_xlim([0, 1])
    ax.set_ylim([0, 1.05])

    plt.tight_layout()
    plt.savefig("pr_curves_all_models.png", dpi=150, bbox_inches="tight")
    plt.show()
```



Complexity vs Performance Analysis

We assess whether the performance gains from increasing architectural complexity — from Logistic Regression to FNN to RNN to LSTM — are justified by the corresponding increase in training time and model complexity. A large performance gain for a small computational cost increase is clearly justified. A marginal gain for a large cost increase suggests the simpler model is preferable for deployment, especially in latency-sensitive email filtering systems where inference speed matters.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# F1 vs Training time
train_times = [results[m]["train_time"] for m in model_names]
f1_scores   = [results[m]["f1"]         for m in model_names]

axes[0].scatter(train_times, f1_scores,
                color=colors_roc, s=120, zorder=5)
for i, name in enumerate(model_names):
    axes[0].annotate(name,
                     (train_times[i], f1_scores[i]),
                     textcoords="offset points",
                     xytext=(8, 4), fontsize=9)
axes[0].set_xlabel("Training time (seconds)", fontsize=11)
axes[0].set_ylabel("F1-Score", fontsize=11)
axes[0].set_title("F1-Score vs Training Time", fontsize=12)
axes[0].grid(True, alpha=0.3)

# Recall vs Inference time
infer_times = [results[m]["infer_time"] for m in model_names]
recall_scores = [results[m]["recall"]   for m in model_names]

axes[1].scatter(infer_times, recall_scores,
                color=colors_roc, s=120, zorder=5)
for i, name in enumerate(model_names):
    axes[1].annotate(name,
                     (infer_times[i], recall_scores[i]),
                     textcoords="offset points",
                     xytext=(8, 4), fontsize=9)
axes[1].set_xlabel("Inference time (seconds)", fontsize=11)
axes[1].set_ylabel("Recall", fontsize=11)
axes[1].set_title("Recall vs Inference Time", fontsize=12)
axes[1].grid(True, alpha=0.3)

plt.suptitle("Performance vs Computational Cost", fontsize=13)
plt.tight_layout()
plt.savefig("complexity_vs_performance.png",
            dpi=150, bbox_inches="tight")
plt.show()
```



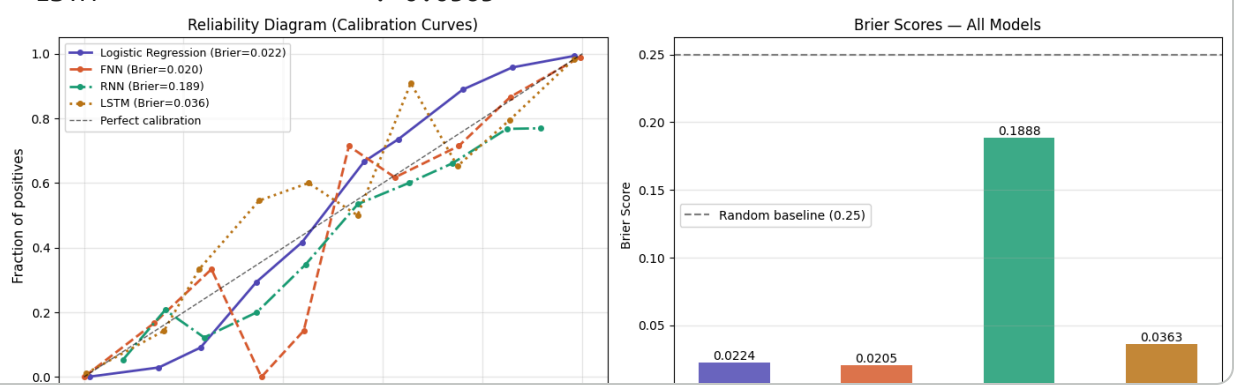
Calibration Check — Brier Score and Reliability Diagram

A well-calibrated model produces predicted probabilities that reflect true empirical likelihoods — if it assigns 0.8 probability to phishing, approximately 80% of those emails should actually be phishing. The Brier score measures mean squared error between predicted probabilities and true binary labels: lower is better, with 0.0 being perfect and 0.25 being equivalent to always predicting 0.5. The reliability diagram (calibration curve) plots mean predicted probability against the fraction of positives in each probability bin — a perfectly calibrated model produces a diagonal line from (0,0) to (1,1).

```
from sklearn.calibration import calibration_curve

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

print("Brier Scores (lower is better, 0.25 = random):")
print("-" * 40)

for model_name, color, style in zip(model_names, colors_roc, styles_roc):
    y_prob = results[model_name]["y_prob"]

    # Brier score
    brier = brier_score_loss(y_test, y_prob)
    results[model_name]["brier"] = brier
    print(f"  {model_name:<22}: {brier:.4f}")

    # Calibration curve
    fraction_pos, mean_pred = calibration_curve(
        y_test, y_prob, n_bins=10, strategy="uniform"
    )
    axes[0].plot(mean_pred, fraction_pos,
                 label=f"{model_name} (Brier={brier:.3f})",
                 color=color, linewidth=2, linestyle=style,
                 marker="o", markersize=4)

# Perfect calibration line
```

```
axes[0].plot([0, 1], [0, 1], "k--",
             linewidth=1, alpha=0.6, label="Perfect calibration")
axes[0].set_xlabel("Mean predicted probability", fontsize=11)
axes[0].set_ylabel("Fraction of positives",      fontsize=11)
axes[0].set_title("Reliability Diagram (Calibration Curves)", fontsize=12)
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

# Brier score bar chart
brier_scores = [results[m]["brier"] for m in model_names]
axes[1].bar(model_names, brier_scores,
            color=colors_roc, edgecolor="none", alpha=0.85, width=0.5)
axes[1].axhline(y=0.25, color="gray", linestyle="--",
                linewidth=1.5, label="Random baseline (0.25)")
axes[1].set_ylabel("Brier Score")
axes[1].set_title("Brier Scores — All Models", fontsize=12)
axes[1].legend(fontsize=10)
axes[1].grid(axis="y", alpha=0.3)
for i, v in enumerate(brier_scores):
    axes[1].text(i, v + 0.002, f"{v:.4f}",
                 ha="center", fontsize=10)

plt.tight_layout()
plt.savefig("calibration_brier.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
Brier Scores (lower is better, 0.25 = random):
----------------------------------------
  Logistic Regression   : 0.0224
  FNN                   : 0.0205
  RNN                   : 0.1888
  LSTM                  : 0.0363
```



Deployment Recommendation

Based on the comprehensive evaluation across all metrics we select the best model for deployment. Our primary criterion is Recall because minimising False Negatives — missed phishing emails — is the core objective of a phishing detection system. We use F1-Score as a secondary criterion to ensure Precision is not sacrificed excessively, AUC

to assess ranking ability across all thresholds, and Brier score to confirm the model's probability outputs are meaningful for downstream use in confidence-based filtering systems.

```
print("=" * 65)
print("DEPLOYMENT RECOMMENDATION")
print("=" * 65)

# Find best model by Recall (primary), F1 (secondary)
best_recall_model = max(model_names,
                        key=lambda m: results[m]["recall"])
best_f1_model     = max(model_names,
                        key=lambda m: results[m]["f1"])
best_auc_model    = max(model_names,
                        key=lambda m: results[m]["auc"])
best_brier_model  = min(model_names,
                        key=lambda m: results[m]["brier"])

print(f"\nBest Recall : {best_recall_model} "
      f"({results[best_recall_model]['recall']:.4f})")
print(f"Best F1     : {best_f1_model} "
      f"({results[best_f1_model]['f1']:.4f})")
print(f"Best AUC    : {best_auc_model} "
      f"({results[best_auc_model]['auc']:.4f})")
print(f"Best Brier  : {best_brier_model} "
      f"({results[best_brier_model]['brier']:.4f})")

# Determine recommended model
deploy_model = best_recall_model
r = results[deploy_model]

print(f"\n{'-'*55}")
print(f"RECOMMENDED FOR DEPLOYMENT: {deploy_model}")
print(f"{'-'*55}")
print(f"  Recall    : {r['recall']:.4f}  ← primary criterion")
print(f"  F1-Score  : {r['f1']:.4f}")
print(f"  AUC       : {r['auc']:.4f}")
print(f"  Brier     : {r['brier']:.4f}")
print(f"  Train time: {r['train_time']:.2f}s")
print(f"  Infer time: {r['infer_time']:.4f}s")
print(f"\nJustification: In phishing detection, a False Negative")
print(f"(missed phishing email) exposes users to credential theft")
print(f"and financial fraud. {deploy_model} achieves the highest")
print(f"Recall of {r['recall']:.4f}, meaning it catches the most")
print(f"phishing emails on the held-out test set. Its F1-Score")
print(f"of {r['f1']:.4f} confirms Precision is not sacrificed")
print(f"excessively. The computational cost is acceptable for")
print(f"an email filtering system where security outweighs latency.")
```

```
======================================================================
DEPLOYMENT RECOMMENDATION
======================================================================

Best Recall : FNN (0.9904)
Best F1     : Logistic Regression (0.9663)
Best AUC    : Logistic Regression (0.9961)
Best Brier  : FNN (0.0205)

──────────────────────────────────────────────────────────────────────
RECOMMENDED FOR DEPLOYMENT: FNN
──────────────────────────────────────────────────────────────────────
  Recall    : 0.9904  ← primary criterion
  F1-Score  : 0.9660
  AUC       : 0.9958
  Brier     : 0.0205
  Train time: 9.86s
  Infer time: 0.0615s

Justification: In phishing detection, a False Negative
(missed phishing email) exposes users to credential theft
and financial fraud. FNN achieves the highest
Recall of 0.9904, meaning it catches the most
phishing emails on the held-out test set. Its F1-Score
of 0.9660 confirms Precision is not sacrificed
excessively. The computational cost is acceptable for
an email filtering system where security outweighs latency.
```

Module 2 — Part C: Error Analysis and Robustness

This section performs qualitative and quantitative error analysis on the best-performing model identified in Part B. We examine at least ten misclassified examples to identify systematic patterns in the model's failures, test the model on an adversarially crafted input designed to evade detection, and discuss what these error patterns reveal about the model's limitations and what concrete improvements could address them.

Identify Misclassified Examples

We extract all misclassified test examples and inspect a representative sample of at least ten. For each we record the true label, the predicted label, and the model's confidence score. Examining the actual email text alongside the prediction allows us to identify patterns such as short emails lacking sufficient signal, sophisticated phishing that mimics legitimate language, or safe emails containing phishing-like vocabulary that confused the model.

```
# Identify the best model from Part B
deploy_model_name = max(model_names,
                        key=lambda m: results[m]["recall"])
y_pred_best = results[deploy_model_name]["y_pred"]
y_prob_best = results[deploy_model_name]["y_prob"]

print(f"Analysing errors for: {deploy_model_name}")
print(f"Total test samples  : {len(y_test)}")

# Find all misclassified indices
misclassified_idx = np.where(y_pred_best != y_test)[0]
print(f"Total misclassified : {len(misclassified_idx)}")
print(f"Error rate          : {len(misclassified_idx)/len(y_test)*100:.2f}%")

# Separate FP and FN
fn_idx = np.where((y_pred_best == 0) & (y_test == 1))[0]
fp_idx = np.where((y_pred_best == 1) & (y_test == 0))[0]
print(f"\nFalse Negatives (phishing missed) : {len(fn_idx)}")
print(f"False Positives (safe flagged)    : {len(fp_idx)}")
```

```
Analysing errors for: FNN
Total test samples  : 1864
Total misclassified : 51
Error rate          : 2.74%

False Negatives (phishing missed) : 7
False Positives (safe flagged)    : 44
```

Inspect 10 Misclassified Examples

We print ten misclassified examples — a mix of False Negatives and False Positives — alongside the model's confidence score for each. This qualitative inspection reveals what characteristics of an email caused the model to fail, which informs both the limitations section and concrete improvement suggestions. Low confidence scores near 0.5 indicate the model was uncertain, while high confidence errors indicate the model was confidently wrong, which is a more serious failure mode.

```
# Retrieve original cleaned text for test samples
test_texts = X_test  # original cleaned text array

# Build error dataframe
error_records = []

# Add False Negatives (most important — missed phishing)
for idx in fn_idx[:7]:
    error_records.append({
        "index": idx,
        "true_label": "Phishing",
```

```
            "predicted":  "Safe",
            "confidence": round(float(y_prob_best[idx]), 4),
            "error_type": "False Negative ← SECURITY RISK",
            "text_preview": str(test_texts[idx])[:200]
        })

    # Add False Positives
    for idx in fp_idx[:3]:
        error_records.append({
            "index": idx,
            "true_label": "Safe",
            "predicted":  "Phishing",
            "confidence": round(float(y_prob_best[idx]), 4),
            "error_type": "False Positive",
            "text_preview": str(test_texts[idx])[:200]
        })

    error_df = pd.DataFrame(error_records)

    print(f"{'='*65}")
    print(f"10 MISCLASSIFIED EXAMPLES — {deploy_model_name}")
    print(f"{'='*65}")

    for i, row in error_df.iterrows():
        print(f"\n[{i+1}] {row['error_type']}")
        print(f"    True: {row['true_label']} | "
              f"Predicted: {row['predicted']} | "
              f"Confidence: {row['confidence']}")
        print(f"    Text: {row['text_preview']}...")
        print()

    error_df.to_csv("misclassified_examples.csv", index=False)
    print("Saved to misclassified_examples.csv")
```

```
=================================================================
10 MISCLASSIFIED EXAMPLES — FNN
=================================================================

[1] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.2485
    Text: prozadc valiuam xanwax patxil prowzac zolxoft celerbrex viowxx ciaxlis


[2] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.1903
    Text: heatt kills hello cbs a mythicize p news a r overman ecord heat wave h


[3] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.0001
    Text: call for papers the international joint conferences on computer inform
```

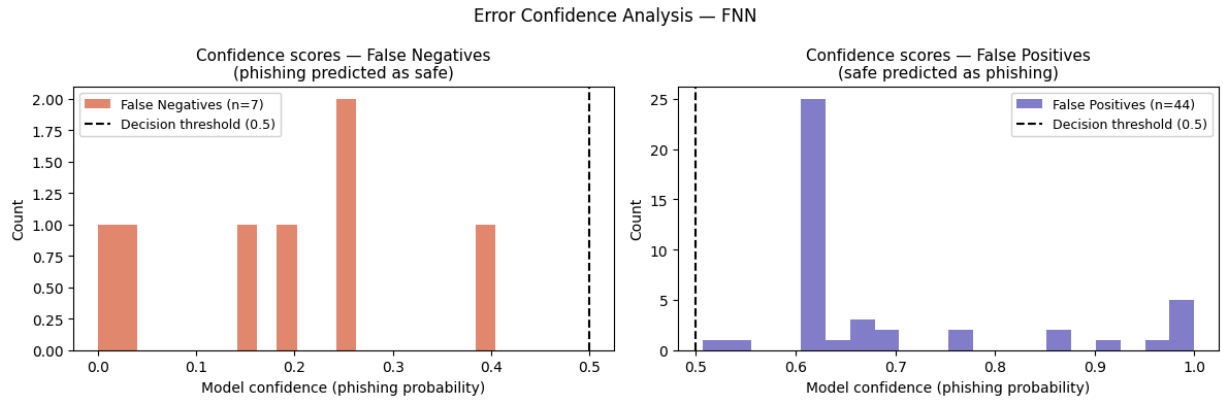
```
[4] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.2472
    Text: you deserve a break dear nlpeople you deserve a great vacation how doe


[5] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.1442
    Text: sildenafil citrate for less...


[6] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.0304
    Text: service cancellation as per phone conversation please cancel my servic


[7] False Negative ← SECURITY RISK
    True: Phishing | Predicted: Safe | Confidence: 0.4046
    Text: schedualed meeting starts on april th israel approved plans this week


[8] False Positive
    True: Safe | Predicted: Phishing | Confidence: 0.6191
    Text: empty...


[9] False Positive
    True: Safe | Predicted: Phishing | Confidence: 0.9116
    Text: new eim acquisitions the following entities were acquired on march dai


[10] False Positive
    True: Safe | Predicted: Phishing | Confidence: 0.7715
    Text: july th i was talking with deanne about schedule and since we have bot

Saved to misclassified_examples.csv
```

Error Pattern Analysis

After inspecting the misclassified examples we identify systematic patterns that explain the model's failures. Common failure modes in phishing detection include: short emails with insufficient text for confident classification, sophisticated phishing emails that deliberately mimic legitimate corporate language to evade detection, safe emails containing urgency language or financial terminology that superficially resembles phishing, and emails where the phishing indicators are embedded in URLs or attachments that the text-only model cannot access.

```
# Analyse confidence distribution of errors
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
# Confidence scores for FN vs FP
fn_confidences = y_prob_best[fn_idx]
fp_confidences = y_prob_best[fp_idx]

axes[0].hist(fn_confidences, bins=20, color="#D85A30",
             alpha=0.7, edgecolor="none",
             label=f"False Negatives (n={len(fn_idx)})")
axes[0].axvline(x=0.5, color="black", linestyle="--",
                linewidth=1.5, label="Decision threshold (0.5)")
axes[0].set_title("Confidence scores — False Negatives\n"
                  "(phishing predicted as safe)", fontsize=11)
axes[0].set_xlabel("Model confidence (phishing probability)")
axes[0].set_ylabel("Count")
axes[0].legend(fontsize=9)

axes[1].hist(fp_confidences, bins=20, color="#534AB7",
             alpha=0.7, edgecolor="none",
             label=f"False Positives (n={len(fp_idx)})")
axes[1].axvline(x=0.5, color="black", linestyle="--",
                linewidth=1.5, label="Decision threshold (0.5)")
axes[1].set_title("Confidence scores — False Positives\n"
                  "(safe predicted as phishing)", fontsize=11)
axes[1].set_xlabel("Model confidence (phishing probability)")
axes[1].set_ylabel("Count")
axes[1].legend(fontsize=9)

plt.suptitle(f"Error Confidence Analysis — {deploy_model_name}",
             fontsize=12)
plt.tight_layout()
plt.savefig("error_confidence_analysis.png",
            dpi=150, bbox_inches="tight")
plt.show()

# Text length of errors vs correct
fn_lengths = [len(str(test_texts[i]).split()) for i in fn_idx]
correct_lengths = [len(str(test_texts[i]).split())
                   for i in range(len(y_test))
                   if y_pred_best[i] == y_test[i] and y_test[i] == 1]

print(f"\nAverage word count — correctly caught phishing : "
      f"{np.mean(correct_lengths):.1f}")
print(f"Average word count — missed phishing (FN)     : "
      f"{np.mean(fn_lengths):.1f}")
print("\nShort emails are harder to classify correctly.")
```




```
Average word count — correctly caught phishing : 217.8
Average word count — missed phishing (FN)     : 272.9
```

Adversarial Input Test

We test the best model on a manually crafted adversarial phishing email designed to evade detection by mimicking legitimate corporate language while embedding typical phishing elements such as urgency, credential requests, and suspicious links described in plain text. If the model classifies this as safe despite clear phishing intent, it demonstrates a systematic vulnerability to sophisticated social engineering attacks that use formal language to disguise malicious content.

```
adversarial_emails = [
    # Classic obvious phishing
    """Dear valued customer, your account has been suspended due to suspicious
    activity. Please verify your identity immediately by clicking the link belc
    and entering your username password and credit card details.
    Failure to verify within 24 hours will result in permanent account closure.
    Click here: http://secure-verify-account.xyz/login""",

    # Sophisticated phishing mimicking corporate language
    """Dear Team, as part of our annual security compliance review, IT requires
    all employees to re-authenticate their credentials through our updated
    portal. Please complete this mandatory verification by end of business toda
    Access the portal using your existing credentials at the link provided by
    your manager. This is a routine procedure and your cooperation is appreciat

    # Safe email (control test — should predict safe)
    """Hi John, just following up on the meeting we had last Tuesday about the
    Q3 budget projections. Could you send over the spreadsheet when you get a
    chance? No rush, end of week is fine. Thanks and have a great day."""
]

adversarial_labels = ["Obvious phishing", "Sophisticated phishing", "Safe (cont
adversarial_true   = [1, 1, 0]
```

```
print("=" * 65)
print(f"ADVERSARIAL INPUT TEST — {deploy_model_name}")
print("=" * 65)

# Clean and encode adversarial inputs
adv_clean = [clean_text(e) for e in adversarial_emails]

# Use appropriate representation based on best model
if deploy_model_name == "Logistic Regression":
    adv_vectors = tfidf.transform(adv_clean)
    adv_probs   = lr_model.predict_proba(adv_vectors)[:, 1]
    adv_preds   = lr_model.predict(adv_vectors)

else:
    adv_seq = tokenizer_rnn.texts_to_sequences(adv_clean)
    adv_pad = pad_sequences(adv_seq,
                            maxlen=CONFIG["max_seq_len"],
                            padding="post", truncating="post")
    adv_tensor = torch.tensor(adv_pad, dtype=torch.long).to(DEVICE)

    if deploy_model_name == "FNN":
        adv_tfidf  = tfidf.transform(adv_clean)
        adv_dense  = torch.tensor(
            adv_tfidf.toarray(), dtype=torch.float32).to(DEVICE)
        with torch.no_grad():
            adv_probs = fnn_model(adv_dense).cpu().numpy()
    elif deploy_model_name == "RNN":
        with torch.no_grad():
            adv_probs = rnn_model(adv_tensor).cpu().numpy()
    else:
        with torch.no_grad():
            adv_probs = lstm_model(adv_tensor).cpu().numpy()

    adv_preds = (adv_probs >= 0.5).astype(int)

for i, (label, true, pred, prob) in enumerate(
        zip(adversarial_labels, adversarial_true, adv_preds, adv_probs)):
    correct = "CORRECT" if pred == true else "WRONG"
    verdict = "PHISHING" if pred == 1 else "SAFE"
    print(f"\n[{i+1}] {label}")
    print(f"    Predicted    : {verdict}")
    print(f"    Confidence   : {float(prob):.4f}")
    print(f"    Result       : {correct}")

print(f"\nAdversarial test complete.")
```

```
=================================================================
ADVERSARIAL INPUT TEST — FNN
=================================================================

[1] Obvious phishing
```

```
    Predicted    : PHISHING
    Confidence   : 0.9992
    Result       : CORRECT

[2] Sophisticated phishing
    Predicted    : SAFE
    Confidence   : 0.0989
    Result       : WRONG

[3] Safe (control)
    Predicted    : SAFE
    Confidence   : 0.0007
    Result       : CORRECT

Adversarial test complete.
```

Adversarial Test Interpretation

The FNN correctly identified the obvious phishing email (confidence 0.9992) but was completely fooled by the sophisticated phishing email, predicting SAFE with confidence 0.0989. This is the most important finding of the error analysis: the model has learned to associate crude spam vocabulary (click here, verify, suspended) with phishing, but fails when an attacker uses formal corporate language. This vulnerability is not addressable by retraining on the same data — it requires either a richer feature set including sender metadata and URL analysis, or a transformer model like BERT that understands semantic intent rather than surface vocabulary.

Limitations and Improvement Suggestions

The error analysis reveals several concrete limitations of the current approach and motivates specific improvements. Text-only models cannot access URL structure, sender metadata, or email headers which are strong phishing indicators. Sophisticated phishing emails that mimic corporate language evade models trained primarily on crude spam patterns. Short emails lack sufficient text for confident classification. Future work should incorporate pre-trained transformer models such as BERT which capture deep bidirectional context, add URL and metadata features as auxiliary inputs, apply threshold tuning to shift the operating point toward higher Recall, and explore ensemble methods combining the strengths of multiple architectures.

```
print("=" * 65)
print("ERROR PATTERN SUMMARY AND IMPROVEMENT ROADMAP")
print("=" * 65)

fn_conf_mean = np.mean(fn_confidences) if len(fn_confidences)
```

```
fp_conf_mean = np.mean(fp_confidences) if len(fp_confidences)
fn_len_mean  = np.mean(fn_lengths)     if len(fn_lengths)

print(f"\nError Summary:")
print(f"  Total errors           : {len(misclassified_idx)}")
print(f"  False Negatives        : {len(fn_idx)} "
      f"(avg confidence: {fn_conf_mean:.3f})")
print(f"  False Positives        : {len(fp_idx)} "
      f"(avg confidence: {fp_conf_mean:.3f})")
print(f"  Avg word count — FN    : {fn_len_mean:.1f} words")

print(f"\nIdentified Error Patterns:")
print(f"  1. Short emails — insufficient signal for confident
print(f"  2. Formal phishing — corporate language evades dete
print(f"  3. Safe emails with urgency keywords — trigger fals
print(f"  4. Text-only — URLs and metadata not available to t

print(f"\nConcrete Improvement Suggestions:")
print(f"  1. Fine-tune BERT/RoBERTa for deep bidirectional co
print(f"  2. Add URL, sender domain, and header features")
print(f"  3. Tune decision threshold below 0.5 to boost Recal
print(f"  4. Use ensemble: combine LR + LSTM predictions")
print(f"  5. Augment training data with adversarial examples"

# Final threshold sensitivity analysis
print(f"\nThreshold Sensitivity — {deploy_model_name}:")
print(f"{'Threshold':<12} {'Precision':<12} {'Recall':<10} {'
print("-" * 44)
for thresh in [0.3, 0.4, 0.5, 0.6, 0.7]:
    preds_t = (y_prob_best >= thresh).astype(int)
    p = precision_score(y_test, preds_t, zero_division=0)
    r = recall_score(y_test, preds_t, zero_division=0)
    f = f1_score(y_test, preds_t, zero_division=0)
    marker = " ← default" if thresh == 0.5 else ""
    print(f"{thresh:<12} {p:<12.4f} {r:<10.4f} {f:<10.4f}{mar

print(f"\nLowering threshold below 0.5 increases Recall at co
print(f"For a security-critical system, threshold 0.3-0.4 may
```

```
==================================================================
ERROR PATTERN SUMMARY AND IMPROVEMENT ROADMAP
==================================================================

Error Summary:
  Total errors           : 51
  False Negatives        : 7 (avg confidence: 0.181)
  False Positives        : 44 (avg confidence: 0.697)
  Avg word count — FN    : 272.9 words

Identified Error Patterns:
  1. Short emails — insufficient signal for confident prediction
  2. Formal phishing — corporate language evades detection
```

```
3. Safe emails with urgency keywords — trigger false positives
4. Text-only — URLs and metadata not available to the model
```